

PHP 基礎の次にやること

PHP の教科書

応用編

久永 勝雄

Eternal Lab

はじめに

私が初めて仕事で使ったプログラミング言語は FORTRAN でした。その後、VB (Visual Basic) などを学び、それらに満足していました。しかし、仕事で WEB 開発を行う必要が生じ、PHP を使うことになりました。最初は仕方なく触れていた PHP ですが、次第にその魅力に引き込まれました。WEB 画面を作成するためには HTML や CSS の知識も必要で、最初は苦労しましたが、WEB の画面遷移の考え方に次第に慣れてくると、こちらの方が普通と感じるようになりました。

さて、PHP5 の時代の教科書は、時代に合わない部分も出てきました。そこで、いくつか新しい方法を取り入れた「PHP の教科書」を作成することにしました。掲示板システムの作成を通じて、段階的に PHP の基本を学べる内容です。

ただ単にソースコードを入力するためではなく、設計に必要な UML、データベース正規形、新しい CSS 手法の flex を使った HTML と CSS、Fetch API を使った JavaScript と PHP の結合を入れています。JavaScript に関しては最近 JQuery が下火になっていることもあり、Vue.js などを使う方法もありましたが、初心者向けに生の JavaScript を使っています。

この教科書を使って学ぶ際には、単にソースコードを入力するだけでなく、その意味をしっかりと理解しながら進んでください。また、教室で使用する場合は、最新の方法があるかもしれませんので、講師のアドバイスを受け入れ、積極的に改良を加えていきましょう。

目次

1. PHP の準備	6
サーバとは	6
Linux とは	6
AMPPS とは	6
Docker とは	7
2. 設計	12
UML で設計	12
データベース正規化	14
■ 非正規形	14
■ 第1 正規形	15
■ 第2 正規形	17
■ 第3 正規形	17
フロー設計	20
画面設計	22
■ ユーザ登録画面	22
■ 登録確認画面	23
■ 登録完了画面	24
■ ログイン画面	25
■ 投稿画面	26
■ ログアウト画面	28
3. データベース作成	30
SQL で作成する	30
■ PhpMyAdmin	30
■ Mysql コマンド	30
4. ユーザ登録サブシステム	36
あらかじめ試しておきたい処理	36
■ データベース接続	37
■ INSERT 文実行	38
HASH 化	40
HTML と CSS のひな型を作る	41
ハートマークなどの記号	46
フォーム情報をチェックする	48
セッション	50
共通ルーチン	57

ユーザ登録.....	59
確認でキャンセルした場合.....	60
登録済みのユーザの時.....	61
登録確認画面.....	62
5. LOGIN サブシステム.....	64
あらかじめ試しておきたい処理.....	64
■ SELECT 文実行.....	64
■ 認証方法.....	65
LOGIN 画面.....	66
6. 投稿画面.....	72
あらかじめ試しておきたい処理.....	72
■ メッセージ書き込み.....	72
■ いいねのための Fetch API.....	72
■ モーダル画面.....	77
投稿フォームの設計.....	82
データ一覧.....	90
メッセージ書き込み.....	100
コメント入力・保存.....	100
削除機能.....	101
メッセージへのいいね.....	103
コメントへのいいね.....	109
7. ログアウト.....	113
セッション削除.....	113
8. Index.....	115

PHP の準備

まずは PHP を実行できる環境を作成していくための章です
いろいろな方法がありますので、やってみましょう

1. PHP の準備

PHP はサーバーサイド言語なので WEB サーバのようなサーバ環境が必要になります。

サーバとは

Visual Studio などですべてのプログラムを作成した場合、プログラムが実行されるのは実行を指示したパソコン自体になります。このようなプログラムをクライアントサイドのプログラムといいます。

それに対して、HTTP のリクエストを受けて実行されレスポンスを返す形のプログラムがサーバサイドプログラムといいます。PHP や javaServlet などがそれにあたります。

そのため、PHP を実行するには HTTP のリクエストを受ける HTTP サーバを構築する必要があります

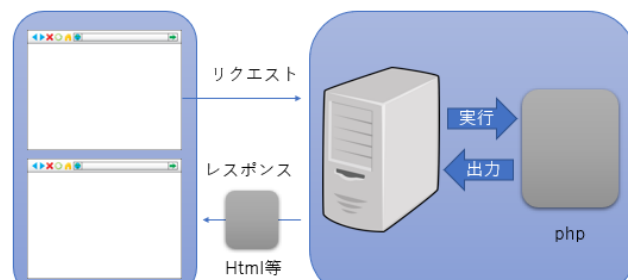


図 1-1 サーバの動き

Linux とは

ネットの検索などをする時にはブラウザを使ってリクエストをサーバに送って、その結果をレスポンスとして見ています。多くの場合、そのサーバは Linux OS で稼働しています。

Linux は、もともと有償であった Unix OS をリーナス・トーバルズ氏がオープンソースで作り直したものでリーナスさんの Unix で Linux といいます。

この本の読者が複数台のパソコンを利用できる環境であれば一台を Linux に、もう一台を Windows で開発すれば 2 台でサーバとクライアントの役目をさせればいいのですが、作業環境作りも操作も難しくなります。

そこで Linux サーバの代わりのものを用意してみます。

AMPPS とは

まず 1 つ目の方法は ampps という Web 系の開発用に特化したツールです。以前は XAMPP を使用していましたが、最近不具合が続くので切り替えました。

AMPPS は、Apache, MySQL, PHP, Perl, Python などの略で、これをインストールすると Linux サーバで使いたいサービスや言語がひとまとまりで使えるようになるのでとても便利なツールです。

<https://ampps.com> からダウンロードでき、メンテナンスも続けられているので初心者の方です。言語のバージョンを切り替えるなどをするには有料版が必要ですが、そのままであれば無料で使えます。またインストール時に mail address を求められますが、必須ではありません。

Docker とは

もう一つの方法は、Docker というコンテナ技術を使った環境で、PHP のためだけでなく、Python や anaconda など linux で稼働するサービスを自前で組み立てていろいろな環境を作れるものです。Windows, Mac, Linux など稼働するので、複数の OS やパソコンで同じ環境を作るのに便利です。少し勉強が必要ですが、決め打ちのものを実行するだけであれば、環境の作成削除が容易にできるので、便利です。

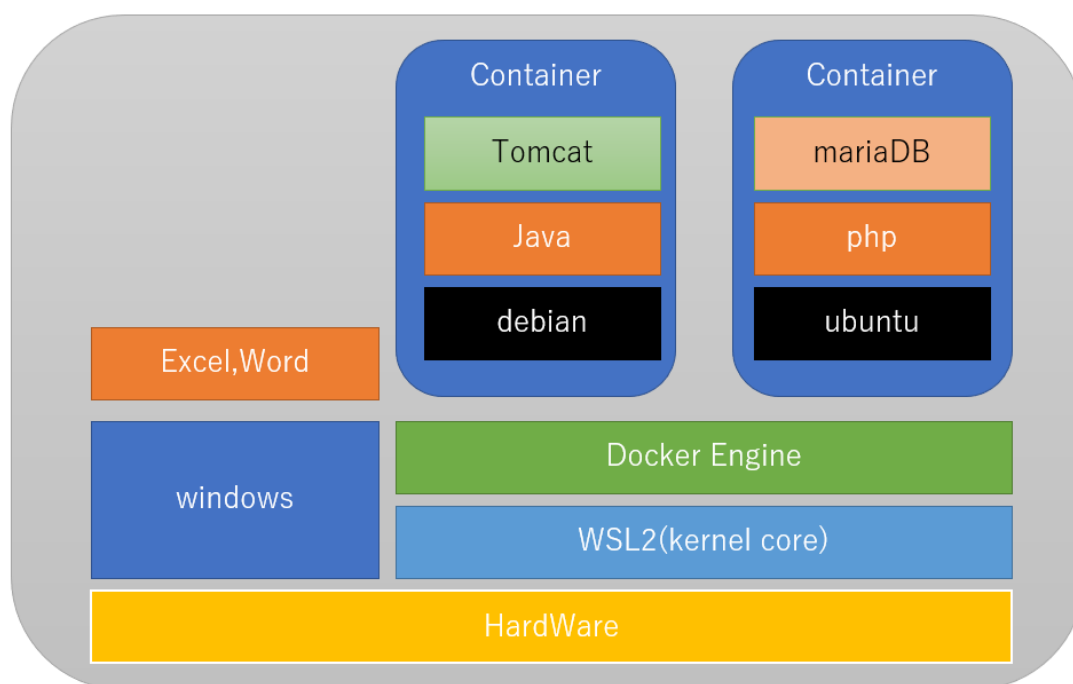


図 1-2 Docker

インストール方法は、www.docker.com から docker desktop をダウンロードしてインストールします。詳しくはネット等で検索してください。(教育用資料-Docker で開発環境など)

Docker を使って AMPPS と同じ環境を作る人のために Docker の宣言ファイルのサンプルを紹介しておきます。phpMyAdmin は id,pass を省略しています。

【compose.yaml】

```
services:
  db:
    image: mariadb:10.7
    environment:
      MARIADB_ROOT_PASSWORD: rootpass
      MARIADB_DATABASE: bbs
      MARIADB_USER: testuser
      MARIADB_PASSWORD: testpass
      TZ: Asia/Tokyo
    volumes:
      - db-data:/var/lib/mysql
  phpmyadmin:
    image: phpmyadmin:5.2
    depends_on:
      - db
    environment:
      PMA_HOST: db
      TZ: Asia/Tokyo
    ports:
      - "8080:80"
    volumes:
      - phpmyadmin-data:/sessions
  apache:
    build: .
    depends_on:
      - db
    container_name: 'apache'
    ports:
      - "8000:80"
    volumes:
      - ./src:/var/www/html
volumes:
  db-data:
  phpmyadmin-data:
```

【Dockerfile】

```
FROM php:8.0-apache
RUN apt-get update && apt-get install -y libonig-dev && docker-php-ext-install pdo_mysql
```

この宣言で AMPPS とほぼ同じ環境を作成できます。

サンプルですので、mariadb を MySQL にしたり、PHP のバージョンを変えるなどしてもかまいません。

このあと、このファイルがあるフォルダーにコマンドプロンプトで移動(cd 等を使う)して以下の命令を入力すると、実行環境が動きます。

```
docker compose up -d --build
```

終了するときは

```
docker compose down
```

としてください。

これからサーバーアプリを作成していきますが、まずは設計をしなければなりません
設計をしないで、ただやみくもにコードを作成してはいけません
正しい設計をすれば後で手戻りが少なくなります

2. 設計

UML で設計

最初になにをするシステムなのか、はっきりさせましょう。

今回作成するのは、掲示板です。複数のユーザがメールアドレス、ニックネーム、パスワード、あれば写真登録してユーザ登録し、ログインができるようにしておきます。

各ユーザはただ一つの掲示板にそれぞれメッセージを書き込んだり、そのメッセージにいいねを入れたり、コメントを書いたりできるようにします。

すると登場人物は、ユーザですね。管理人は今回は作りません。

できることは、ユーザ登録、ログイン、メッセージの書き込み、削除、いいねを押す、コメントを書く、コメントにいいねを入れるです。

各ユーザは、一つのメッセージに対して 1 回だけいいねを押せます。いいねはいいねボタンを押して行います。また解除もできます。解除はもう一度いいねボタンをクリックします。メッセージに対して何人からいいねが付いたかその数が表示されます。ログアウトしてもう一度ログインした時、自分がいいねを入れたメッセージはハートが赤く塗りつぶされて表示されます。

他人のメッセージを消したり、編集したりすることはできません。

メッセージにはコメントを書くことができます。一つのメッセージに複数のコメントを書くことができます。またコメントに対しても各ユーザが 1 回だけいいねをいれることができます。解除することもできます。いいねを入れたコメントは、再度参照したときハートが赤く塗りつぶされて表示されます。またコメントは削除できません。他のユーザが何人いいねを入れたかはわかりません。

せっかくですので UML(Unified Modeling Language)統一モデリング言語を少し使って設計をやってみましょう。

UML のユースケース図を使ってみましょう。

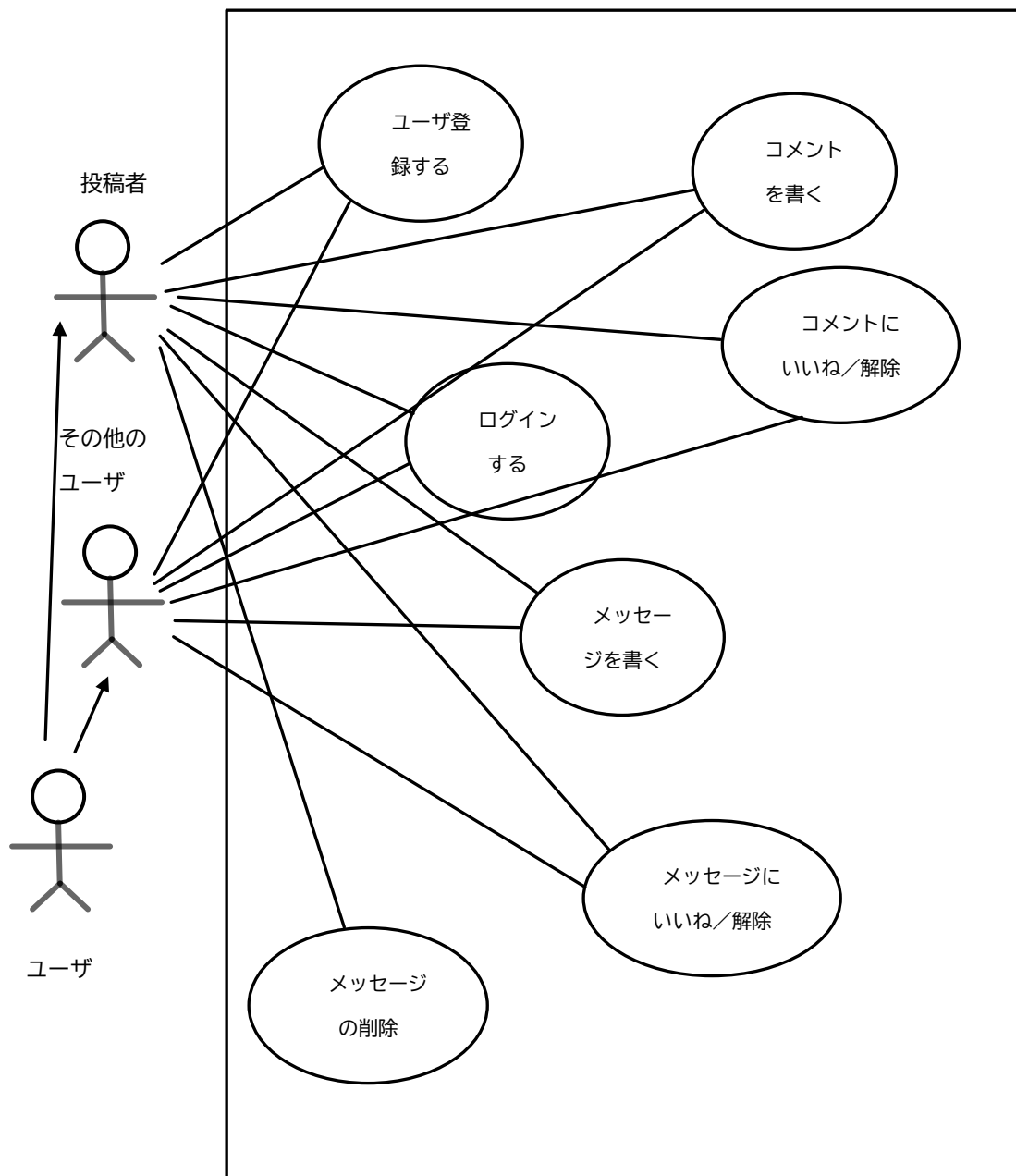


図 2-1 ユースケース図

これがユースケース図です。左側の人の形がユーザを表します。ユーザは役割として投稿者とその他のユーザに分類されます。それぞれから伸びている線がその役割ができることを表しています。

大きな四角の中が、このシステムで出来ることです。一つの丸の中に一つの出来ることが書いてあります。ほとんどのことはどのユーザもできますが、その投稿者でない人はそのメッセージを削除することができません。

このような図にすると、プログラマでない人でも、システム上で誰に何ができて、何ができないか分かるので、仕様の確認などで使用できます。

データベース正規化

■ 非正規形

まずは、必要なデータをリストアップしてみましょう

メンバー名、email、password、picture、作成日、更新日
メッセージ、メッセージ作成日付、メッセージ更新日付、メッセージへのいいね、コメント、コメントへのいいね

図 2-2 メンバー一覧

このようなデータが1つのレコードにまとまっているテーブルがあったら、重複するデータや、無駄なデータがいっぱい出来てしまいますね。この状態を「非正規形」といいます。たとえば Excel の1行に関連する情報を全部書いてしまった状態です。

メンバー一覧の1行目のデータは、メンバーに関するもので、ひとまとめにしてもよさそうです。本来はこの分割は第3正規形（後述）で行います。

項目名	メンバー ID	メンバー 名	email	password	picture	作成日	更新日
名称	mid	mname	email	pass	picture	created	modified
型	Int 主キー	varchar 255	varchar 100	varchar 100	varchar 255	datetime	datetime

図 2-3 メンバー表

さて問題は2，3行目です。これらを一つのテーブルにまとめていいでしょうか？このようなテーブルを作った場合、非常に非効率になります。

まず、非正規形を書いてみます。

メッセージ	メンバー名、 email, password, など	作成日付	更新日付	メッセージ のいいね	コメント	コメントの いいね
-------	----------------------------------	------	------	---------------	------	--------------

■ 第1正規形

最初に、繰り返しを含む部分を分割します。これを「第1正規形」といいます。

メッセージ	メンバー名、 email, password, など	作成日付	更新日付	メッセージ のいいね	コメント	コメントの いいね
-------	----------------------------------	------	------	---------------	------	--------------

一つのメッセージに対して、いいねは複数のユーザから来る可能性があるので、メッセージのいいねは分割します。分割の候補の左側に太い縦棒を入れました。そして一つのメッセージに対してコメントが複数くる可能性があるのもこれも分割します。そのそれぞれのコメントに対していいねが各ユーザから来る可能性があるのも分割します。

ということで、5つのテーブルに分解します。ここでメンバー名に関してですが、同姓同名の場合区別できないので、先に作ったメンバー表のメンバーIDを使うようにします。

・メンバー表

メンバーID	メンバー名	Email	Password	写真	作成日	更新日
--------	-------	-------	----------	----	-----	-----

・メッセージデータ

メッセージ	メンバー ID(FK)	作成日付	更新日付
-------	----------------	------	------

ここでメンバーID はメンバー表の主キーを指すキーなので外部キー（以降、表中では FK と略すかオレンジ色にします）といいます。

・メッセージのいいねデータ

メッセージのいいね

・コメントデータ

コメント

・コメントのいいねデータ

コメントのいいね

しかし、ただ分解してだけでは関係が切れてしまいます。関係が切れると、コメントだけ見ても、それがどのメッセージ対するものか、分からなくなります。そこでリンクを張ります。

メッセージのいいねデータは、どのメッセージへの、誰のいいねか分かるように、メッセージ ID とメンバーID をつけましょう。また主キーも付けておきましょう

・メッセージのいいねデータ

メッセージいいね ID (主キー)	メッセージ ID	メンバー名
-------------------	----------	-------

今度は、コメントです。

コメントは各ユーザがメッセージに対していくつでも書き込むことができるので、どのメッセージに誰がいつどの順番でコメントを入れたかがわかるようにする必要があります。

どの順番の部分はオートインクリメント¹ (以降、表中で AI と略す) という方法を使います。コメントを書き込む際に最終のコメントの番号を調べて、+ 1 の番号でコメントを書き込もうとしたとします。ところが、同じタイミングで最終番号を読み込み、+ 1 で書き込もうとする別のユーザが居た場合、番号がぶつかってしまいます。そこでプログラムでは番号を付けず、データベース側に自動で番号を振らせます。これならば一瞬でも早く書き込んだ方が先の番号になります。

・コメントデータ

コメント ID (主キー) AI	メッセージ ID	メンバー名	コメント	作成日
---------------------	----------	-------	------	-----

¹ この呼び名は MySQL のもので、SQLServer では IDENTITY と呼びます

今度はこのコメントに対するいいねを考えましょう。

こちらでもオートインクリメントを使いましょう。どのコメントに誰がいいねを入れたかがわかるようにしましょう。

・コメントいいねデータ

コメントいいね ID(主キー)	メッセージ ID	メンバーID
AI		

さて、肝心のメッセージデータです。

ここでもオートインクリメントを使いましょう。そして誰がいつ何を書いたかわかるようにしましょう。

・メッセージデータ

メッセージ ID (主キー)	メッセージ	メンバーID	作成日	更新日	削除フラグ
AI					

これでようやく第1正規形が終わりました。

■ 第2正規形

第2正規形は主キーになっている項目の一部だけに従属する項目の分割です。たとえば、購入履歴データに商品のデータが入っていた場合、商品名と価格などが履歴データに入っているとその商品を購入するたびに同じ価格データが入って無駄です。そこで商品に関するデータを別テーブルに分割して、商品IDを履歴データに入れて、商品データとリンクすると無駄がなくなります。このようなことをするのが第2正規形ですが、今回は無いようです。

■ 第3正規形

第3正規形は主キー以外に主従関係がある項目を分割し、計算で求められるものを削除することです。

今回は、メンバー表のメンバー名がこれに当たりますが、最初にやってしまいました。非正規形の中でメンバー名、email、password、pictureがこれに当たります。これが各レコードに存在すると無駄です。で、メッセージデータやいいねデータの中にはメンバーIDを入れて、メンバー表とリンクしています。

では、今回分解したテーブルを全部書き出して見ましょう。

【メンバー表(members)】

項目名	メンバーID	メンバー名	email	password	写真	作成日	更新日
名称	mid	mname	email	pass	picture	created	modified
型	Int 主キー・AI	varchar 255	varchar 100	varchar 255	varchar 255	datetime	timestamp

写真は写真を保存したパス名を入れます。

【メッセージデータ(messages)】

項目名	メッセージ ID	メッセージ	メンバーID	作成日	更新日	削除フラグ
名称	meid	message	mid	mecre	memod	deleted
型	int 主キー・AI	varchar 255	int FK	datetime	timestamp	boolean

メッセージを消した場合、このレコードを消してしまうと、このレコードへのリンクが宙ぶらりんになってしまうので、削除しないで削除フラグを True にします

【コメントデータ(comments)】

項目名	コメント ID	メッセージ ID	メンバーID	コメント	作成日
名称	cid	meid	mid	comment	cocre
型	Int 主キー・AI	Int FK	Int FK	varchar 255	datetime

【メッセージいいねデータ(melike)】

項目名	メッセージ ID	メンバーID
名称	meid	mid
型	Int (主キー)	Int (主キー)

同じメッセージに同じ人が2回いいねができないように、2つのキーの組み合わせを主キーとします。

【コメントいいねデータ(colike)】

項目名	コメント ID	メンバーID
名称	cid	mid
型	Int (主キー)	Int (主キー)

同じコメントに同じ人が2回いいねができないように、2つのキーの組み合わせを主キーとしました。

データベースの正規化ができれば、プログラムは出来上がったようなものです。逆に言うところ間違っていると最初からやり直しになります。正しい正規化ができることが SE²への近道です。

² SE(System enginer、システムエンジニア)

フロー設計

作るべき画面をリストアップしましょう。

- ユーザ登録画面
- 登録確認画面
- 登録完了画面
- ログイン画面
- 投稿画面
- ログアウト画面

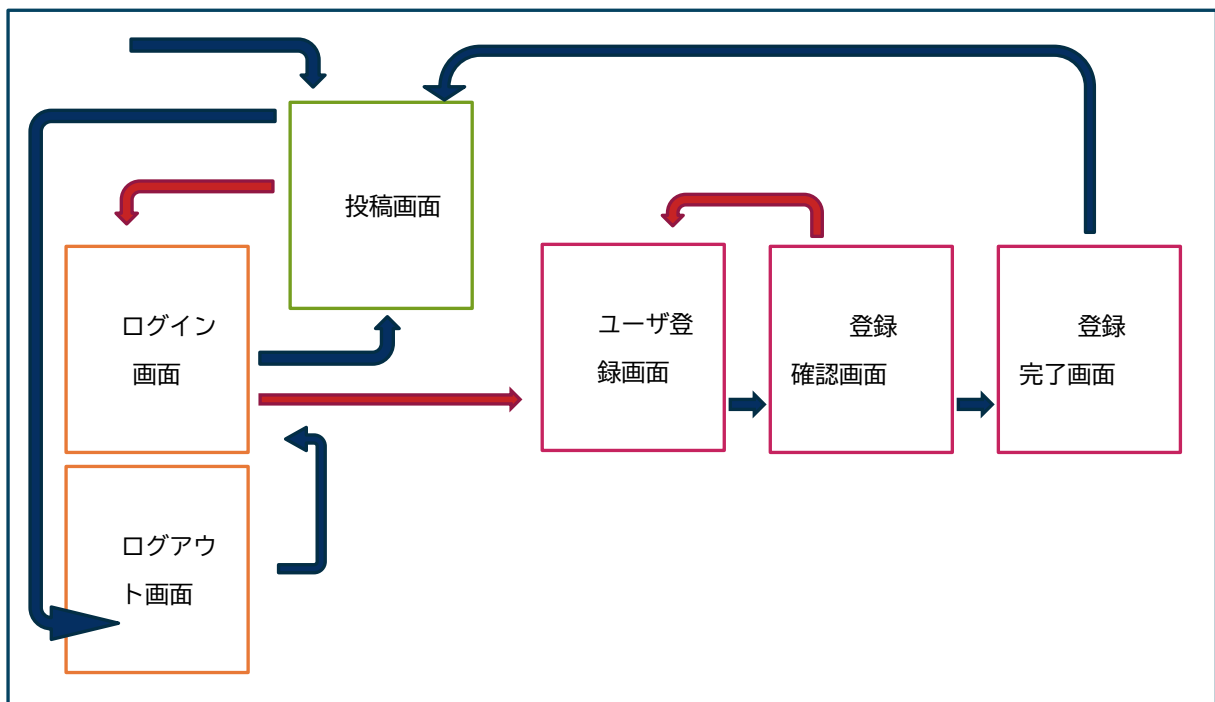


図 2-4 処理フロー図

1. 最初のアクセスは投稿画面に行きます
2. ログインできていないとログイン画面にジャンプします
3. ユーザであればログインして投稿画面にジャンプしますが
4. ユーザ登録をしていない場合はユーザ登録画面にジャンプします
5. ユーザ登録の確認画面で正しくない場合はユーザ登録画面へ戻ります
6. 登録確認ができたなら登録完了画面にジャンプします
7. 再度登録画面に飛びますがログインしていないので再びログイン画面にジャンプします
8. ログイン画面でログインできれば投稿画面にジャンプします
9. 投稿が終わったらログアウトします

このフローで作成しますが、登録系のものはサブシステムとして一つ下のフォルダーに作ります。

ではファイルの階層を図にしてみます

```
/phpsample
  index.php(投稿画面)
  login.php(ログイン画面)
  logout.php(ログアウト画面)
  +--/join
    index.php(ユーザ登録画面)
    check.php(登録確認画面)
    complete.html(登録完了画面)
  +--/css
    style.css(スタイルシート)
```

図 2-5 ファイル階層

画面設計

画面の設計をしてみましょう。ここでは、まだ HTML/CSS は使いません

これらの大体の設計図を考えます。レスポンスで考えますので、モバイル画面と PC 画面の 2 種類考えます。

■ ユーザ登録画面

ユーザ登録画面では「ニックネーム」「メールアドレス」「パスワード」「アイコン写真」を登録してもらいます。

モバイル画面

PC 画面

The image displays two wireframes for a user registration interface. The left wireframe is for a mobile screen, showing a vertical layout with a title 'ユーザ登録画面' (User Registration Screen) at the top, followed by input fields for 'Nick Name', 'Mail Address', and 'Password'. Below these is a 'ファイル選択' (File Selection) button, and at the bottom is a large '確認' (Confirm) button. The right wireframe is for a PC screen, showing a similar layout but with a more compact design. It also has the title 'ユーザ登録画面' at the top, followed by input fields for 'Nick Name', 'Mail Address', and 'Password'. Below these is a 'ファイル選択' (File Selection) button, and at the bottom is a '確認' (Confirm) button. The PC version uses a more rounded rectangle for the main form area.

図 2-6 ユーザ登録画面

■ 登録確認画面

登録確認画面では「ニックネーム」「メールアドレス」「パスワード」「写真ファイル名」を確認します。

モバイル画面

PC 画面

The image displays two versions of a registration confirmation screen. The left version is for a mobile device, showing a rounded rectangle with the title '登録確認画面' and four lines of text: 'ニックネーム : tanosan', 'メールアドレス: tano@abc.com', 'パスワード: 1234567', and 'ファイル名: tano1.jpg'. Below the text are two buttons: a dark blue '確認' button and an orange '書き直す' button. The right version is for a PC, showing a similar rounded rectangle with the same title and text, but with the buttons stacked vertically: a dark blue '確認' button on top and an orange '書き直す' button on the bottom.

登録確認画面
ニックネーム : tanosan
メールアドレス: tano@abc.com
パスワード: 1234567
ファイル名: tano1.jpg
確認
書き直す

図 2-7 登録確認画面

■ 登録完了画面

登録完了画面ではニックネームの登録完了を確認してログイン画面に進みます。

モバイル画面

PC 画面



図 2-8 登録完了画面

■ ログイン画面

ログイン画面では「メールアドレス」「パスワード」を入力してログインします。

モバイル画面

PC 画面

The image displays two wireframes for a login interface. The left wireframe represents a mobile screen, featuring a rounded rectangle containing the title 'ログイン画面', a link 'ユーザでない方は登録へ', and input fields for 'Mail Address' and 'Password', followed by a dark blue 'ログイン' button. The right wireframe represents a PC screen, showing a larger rounded rectangle with the same title and link, but with more spacious input fields for 'Mail Address' and 'Password', and a larger 'ログイン' button.

図 2-9 ログイン画面

■ 投稿画面

投稿画面では「メッセージ投稿」「コメント入力」「メッセージへいいね」「メッセージへのいいねの解除」「コメントへいいね」「コメントへのいいねの解除」「メッセージの削除」「ログアウト」ができます。

モバイル画面

PC 画面



図 2-10 投稿画面

投稿画面では、コメントの入力ができます。その際、モーダル画面（終了しないとほかの入力ができない画面）にして、現在の画面の上に半透明の膜を張り、入力できるようにします。

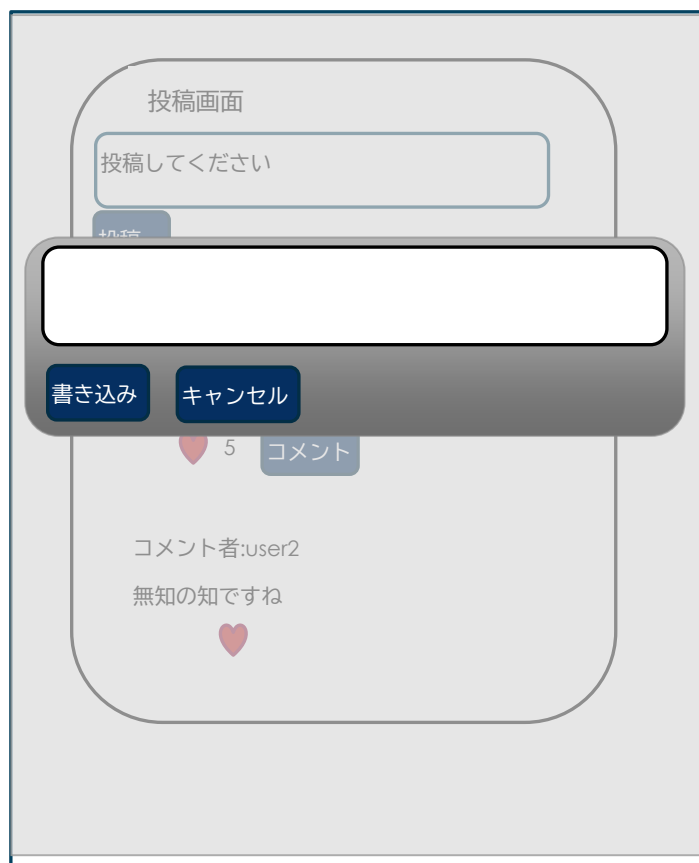


図 2-11 コメント画面

書き込みボタンを押すか、キャンセルボタンを押さない限り画面の外はクリックできません。

書き込みを押すとコメントがメッセージに追加され、画面全体が再表示されます。

キャンセルした場合は、モーダルが解除されます。

また、右上の矢印でログアウトができます。

■ ログアウト画面

ログアウトは、投稿画面の右上のログアウトで行います。logout.php は作りますが、画面は表示しません。

データベース作成

前の章で設計をしたデータベースを作成していきましょう
作成では SQL というデータベースと対話する言語を使用します

3. データベース作成

SQL で作成する

設計の第3正規形で計画したように、データベースを作成していきます。作成するのは5つのテーブルで、MySQL(mariaDB)で作成します。

SQL 文の復習もかねてコマンドで作っていきたいと思います。

■ PhpMyAdmin

phpMyAdmin は、コマンドを使わなくても、マウス操作でデータベース操作ができるようにしたツールで、無料で配布されています。今回は、このツールを使いますが、できるだけ SQL コマンドを理解して欲しいので、マウス操作をできるだけ使わずにデータベースやテーブルを作成していきます。またわかる人は mysql をインストールしてコマンドで操作しても構いません。

AMPPS を利用すると、デフォルトで root 特権を持ったユーザでログインできるようになっています。このアカウントで操作を行うとすべてのことができますが、docker などを使ってユーザログインした場合は、特権がないのでデータベースが作成できなかったりしますので、compose.yaml など进行调整するか、MySQL コマンドを操作するなどしてデータベースが作成できるようにして以降の処理をするようにしてください。以降は特権があることが前提になっています。

■ Mysql コマンド

◇ データベース領域

テーブルを作成する前に、これらのテーブルをまとめるデータベース領域を作成します。phpMyAdmin を起動して SQL を入力して作成します。

```
create database bbs;
```

これを実行すると、新たなテーブル保存先として bbs データベースができます。MySQL コマンドで作製した方は

```
use bbs;
```

として、デフォルトのデータベースをログインのたびに指定してください。phpMyAdmin の方は、左のデータベース一覧から bbs を選択してから次の作業をしましょう。

また、Docker で db を宣言する場合は DATABASE を宣言して作ってしまえば、この処理は必要ありません。

◇メンバー表 (members)

ユーザ登録した人の情報を保存するテーブルをメンバー表として作成します。主キーは自動採番、pass は HASH 化するため長くとります。

```
create table members (  
    mid int not null AUTO_INCREMENT,  
    mname varchar(255) not null,  
    email varchar(100) not null,  
    pass varchar(255) not null,  
    picture varchar(255),  
    created datetime,  
    modified timestamp default current_timestamp on update current_timestamp,  
    primary key(mid)  
);
```

phpMyAdmin で実行する場合は、以下のようになります

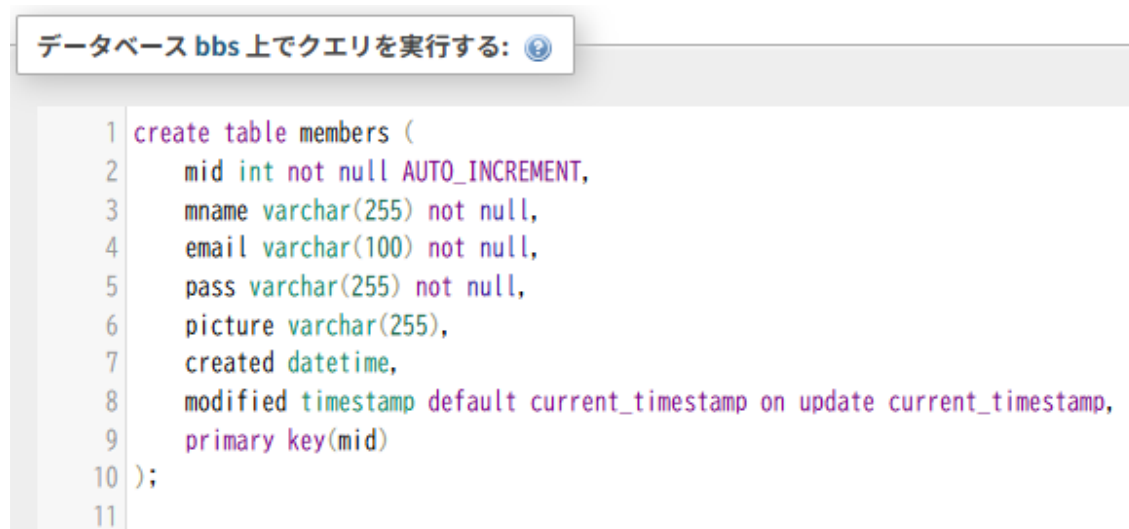


図 3-1 SQL 実行画面

実行した結果は

← サーバ: db > データベース: bbs > テーブル: members									
表示 構造 SQL 検索 挿入 エクスポート インポート 操作 トリガ									
テーブルの構造 リレーションビュー									
#	名前	タイプ	照合順序	属性	Null	デフォルト値	コメント	その他	
☐ 1	mid	int(11)			いいえ	なし		AUTO_INCREMENT	
☐ 2	mname	varchar(255)	utf8mb4_general_ci		いいえ	なし			
☐ 3	email	varchar(100)	utf8mb4_general_ci		いいえ	なし			
☐ 4	pass	varchar(255)	utf8mb4_general_ci		いいえ	なし			
☐ 5	picture	varchar(255)	utf8mb4_general_ci		はい	NULL			
☐ 6	created	datetime			はい	NULL			
☐ 7	modified	timestamp			いいえ	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP()	

図 3-2 構造画面

members の構造を見ると、このようになっていると思います。以下のテーブルも同じように操作してください。

◇メッセージデータ(messages)

主キーは自動採番で、メッセージを保存します。どのメンバーの記述かはメンバーID を保存します。レコードの削除は行わず、deleted フラグを立てます。

```
create table messages (
  meid int not null AUTO_INCREMENT,
  message varchar(255) not null,
  mid int not null,
  mcre datetime not null,
  modified timestamp default current_timestamp on update current_timestamp,
  deleted Boolean not null DEFAULT false,
  primary key (meid)
);
```


◇コメントデータ(**comments**)

メッセージに対して書き込んだコメントを保存します。コメントにコメントはできません。主キーまたは作成日付で順を把握します。コメントは編集・削除できません。

```
create table comments (  
  cid int not null AUTO_INCREMENT,  
  meid int not null,  
  mid int not null,  
  comment varchar(255) not null,  
  cocre datetime,  
  primary key (cid)  
);
```

◇メッセージいいねデータ(**melike**)

メッセージに対するいいねを保存します。どれがどのメッセージにいいねを入れたかを保存します。

```
create table melike (  
  meid int not null,  
  mid int not null,  
  primary key (meid,mid)  
);
```

◇コメントいいねデータ(**colike**)

コメントに対するいいねを保存します。どれがどのコメントにいいねを入れたかを保存します。

```
create table colike (  
  cid int not null,  
  mid int not null,  
  primary key (cid,mid)  
);
```


ユーザ登録サブシステム

システムを作成する時に、そのシステムを小さな機能のかたまりに分けて作ります
ここでは、最初に使用することになるユーザ登録のサブシステムを作っていきます

4. ユーザ登録サブシステム

あらかじめ試しておきたい処理

ここから徐々にソースコードを作りこんでいきましょう。

今回作成するソースコードを、いくつかのサブシステムに分けて作るので、まずは最下層にあるユーザ登録サブシステムから作ります。

```
/phpsample
index.html
+--/join
    Index.php
    check.php
+--/css
    style.css
+--/member_image
```

図 4-1 ファイル階層

AMPPS の場合は、AMPPS フォルダ配下の www フォルダ配下に phpsample とした場合、その下層に join(ユーザ登録用)と css(スタイルシート用)のフォルダを作り、最上位に index.html を置きます。AMPPS の apache を起動して、ブラウザで localhost で実行できます。

```
/phpsample
compose.yaml
Dockerfile
+--/src
    index.html
    +--/join
        Index.php
        check.php
    +--/css
        style.css
    +--/member_image
```

図 4-2 Docker ファイル階層

サンプルの Docker の yaml で言えば src フォルダ配下に同様の階層を作ります。

このようにして Docker を起動しておけば、ソースを保存するだけで apache の環境と同期されて、ブラウザで localhost:8000 で実行できます。

■ データベース接続

さきほど作ったデータベースに対して PHP プログラムから接続していきましょう。PHP のデータベース接続は PDO クラスというデータベース接続用の汎用クラスがあるので、これで接続します。PDO クラスを使えば、MySQL、mariaDB、postgreSQL、Oracle などにわずかな変更で接続できるので、プログラムを共通化できます。

まずは、このデータベースに PHP から接続するところだけで実験をしてみましょう。

【databasetest.php】

```
$host = 'localhost';
$dbname = 'bbs';
$username = 'testuser';
$password = 'testpass';
$charset = 'utf8';
$dns = 'mysql:host='.$host.';dbname='.$dbname.';charset='.$charset;
try {
    $db = new PDO($dns, $username, $password);
    echo '接続できました';
} catch (PDOException $e) {
    echo '失敗しました';
}
```

AMPPS で実験する場合は HOST として自分自身を示す「localhost」または「127.0.0.1」を指定します。別にサーバを作った場合はその名前か IP アドレスを入れてください。また Docker で行った場合はデータベースのコンテナ名（1 章の compose.yaml でいえば db）を指定してください。

\$dbname は create database で作ったデータベース領域名を使います。今回のデータベースは bbs です。この接続はデータベースに対するものでテーブルに対するものではありません。

■ INSERT 文実行

先ほどの databasetest.php を改造して仮のデータを挿入するプログラム作成しましょう。Docker で行った例です。ユーザ名は testuser パスワードは testpass にしていますが、別のものでも構いません。

【databasetest.php】

```
<?php
try {
    $dsn = 'mysql:host=db;dbname=bbs;charset=utf8';
    $db = new PDO($dsn, 'testuser', 'testpass');
    echo "接続に成功しました";
} catch (PDOException $e) {
    echo "接続に失敗しました";
    echo $e->getMessage();
    exit();
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>db 接続</title>
</head>
<body>
<?php
$name = 'test';
$email = 'test@abc.com';
$pass = '12345';
try {
    $statement = $db->prepare('insert into members (mname,email,pass,created)
values(?,?,?,now());');
    $statement->bindParam(1, $name,PDO::PARAM_STR);
    $statement->bindParam(2, $email,PDO::PARAM_STR);
    $statement->bindParam(3, $pass,PDO::PARAM_STR);
    $statement->execute();
    echo "Insert 成功";
} catch(PDOException $e) {
    echo "Insert 失敗";
}

$statement = null;
$db = null;
?>
</body>
</html>
```

PHP と HTML を混合して記述する場合は、<?php と ?>に挟んで記述します。Prepare は SQL インジェクショ

ンという SQL に対して有害な文字列を送り込む攻撃に備えるための手段です。このようにテンプレートを作って、必要な数値や文字列だけ送りこむと、有害文字をはじいてくれます。また \$statement や \$db などのオブジェクトは null を代入して削除できますが、すぐ終了するのであれば、そのまま終了しても大丈夫です。

実行後には、messages テーブルに、このようにデータが挿入されます。

	mid	mname	email	pass	picture	created	modified
☐ 編集 ☐ コピー ☐ 削除	1	test	test@abc.com	12345	NULL	2025-01-24 04:24:27	2025-01-24 04:24:27

図 4-3 メンバー表内容

HASH 化

データベースに接続して、メンバー情報を書き込みましたが、見てわかるようにパスワードが丸見えです。もしもこのテーブルの内容を見ることができたら、セキュリティは無いのと同じです。

なので、データベースにパスワードを保存する際には HASH 化するのが常識です。暗号ではありません。暗号ならば復元する必要がありますが、これはそのまま使用します。暗号化するということは、覗き見ることができる可能性があります。HASH 化したものから元に戻すことはできません。

登録時パスワードの文字=>HASH 化=>HASH 化文字=>これを保存

ログイン時のパスワード文字=>HASH 化=>保存文字列と比較=>同じならば認証

`password_hash()`関数を使うと、与えられた文字列に対してランダムソルトという文字が加えられてから HASH 化されます。

そこだけ変更してみましょう

【databasetest.php 変更部分だけ】

```
$motopass = '12345';  
$pass = password_hash($motopass,PASSWORD_DEFAULT);
```

ただ HASH 化関数を呼ぶだけです。

これを複数回保存してみましょう。すると、元は同じパスワードなのに、別の文字列として保存されます。これにより、解析がむずかしくなります。

```
3 test      test@abc.com $2y$10$w2WD.VxVu7djr6.iuWxbFO/4Xxu8qKu3UTQWcBNE3dL...  
4 test      test@abc.com $2y$10$/ZFZ/IRI/rvJiOouUash9O8q2REQVQK9ZPISXgcFxt/...
```

図 4-4 HASH 化のサンプル

HASH 化は一文字でも違えば大きく変化するのが特徴なので、ランダムソルトという調味料によってこのように大きく変わります。ログイン時は逆の関数 `password_verify()` で認証します。また `PASSWORD_DEFAULT` というオプションが指定されていますが、これはどの程度の難しさで HASH 化するかを現在の PHP のデフォルトに任せるというものです。現在のデフォルトは BCRYPT なので 60 文字になりますが、一応 255 文字まで余裕を持たせています。

HTML と CSS のひな型を作る

さて今までの HTML は PHP を実行するのに必要な部分だけ作成してきましたが、PHP を実行することは、HTML を出力するということなので HTML を理解していないとうまくいきません。

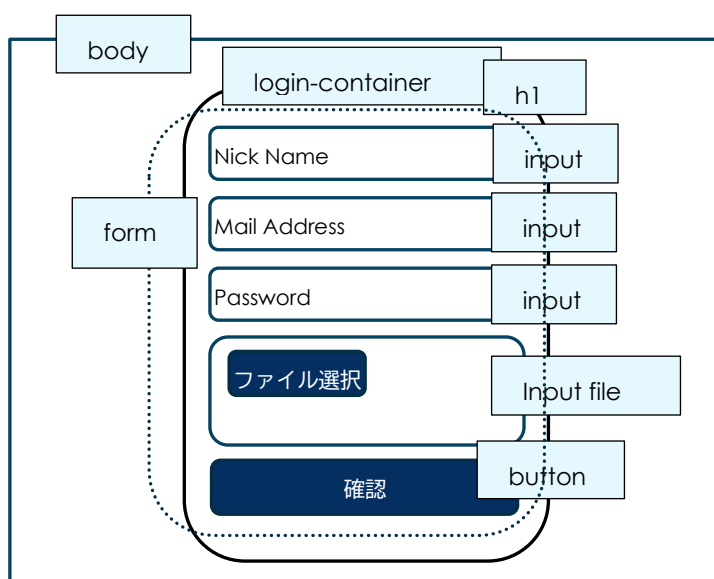
そこで、HTML と CSS のひな型を先に作りましょう。

ユーザ登録の画面ですので図 2-5 に従って作っていきます。HTML で設計することは矩形の領域に区切って、その階層を組み上げることです。

body の中に、login-container が
あり、その中に form があり、form
の部品として input が縦に並んでい
るように配置します。

このように設計したら、HTML を同
じように作っていきます。

実験が終わったので index.php を
正式に作ります。



【join/index.php】

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>会員登録画面</title>
  <link rel="stylesheet" href="../css/style.css">
</head>
```

```
<body>
  <div class="login-container">
    <h1>会員登録画面</h1>
    <form action="" method="post" enctype="multipart/form-data">
      <input type="text" placeholder="Nick Name" name="name" size="35"
minlength="4" maxlength="100" required>
      <input type="email" placeholder="Mail Address" name="mail" size="80"
required>
      <input type="password" placeholder="Password" name="pass" size="20"
minlength="8" maxlength="20" required>
      <input type="file" class="joinFile" name="image" size="120">
      <button type="submit">確認</button>
    </form>
  </div>
</body>
</html>
```

ニックネームは最小４文字、最大１００文字必須。メールアドレスは簡易チェック付きの入力で必須です。パスワードは最小８文字、最大２０文字必須。ファイル名は必須ではありません。ファイル名の送信のために form に enctype を付けています。これを忘れるとファイル送信できません。action は無指定なので index.php 自身が post で受信します。

今度は、CSS です。図 4-1 のように CSS フォルダーを作り style.css を作ります。

【css/style.css】

```
@charset "utf-8";

body {
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    font-family: Arial, sans-serif;
    min-width: 375px;
}

/* Login 用 */
.login-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 20px;
    border: 1px solid #ccc;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    width: 300px; /* デフォルトの幅 */
}

.login-container h1 {
    font-size: 24px;
    text-align: center;
}

.login-container input {
    margin: 10px 0;
    padding: 10px;
    width: 93%;
    border: 1px solid #ccc;
    border-radius: 5px;
}

.login-container button {
    opacity: 0.8;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    background-color: #495aa5;
    color: white;
    cursor: pointer;
    width: 100%;
    margin-bottom: 3px;
}
```

```
.login-container button:hover {  
  opacity: 1.0;  
}  
.login-container input[type="file"]::file-selector-button {  
  background-color: #495aa5;  
  color: #ccc;  
  margin-bottom: 40px;  
  border-radius: 4px;  
}
```

以前との大きな違いは display:flex です。display を指定しないときは div や form は block になり、縦に並びます。Flex を使用するとこれらは横に並びます。しかし今回は flex-direction を使ってこれを縦に並べています。Flex は非常に便利なので、できるだけ flex で作っていきます。それは float を使わなくてもよくなるからで、float の短所の float を切らないと、どこまでも続いてしまうという処理がいらなくなるからです。

float はデザインを難しくしています。flex を使うとそれがたいへん楽になります。Flex の詳しい使い方はデザインの本やネットに譲ります。

さて CSS はこれで終わりではありません。実際のネット環境では、パソコンの広い画面で見る人よりも、スマホの狭い画面で見る人の方が圧倒的に多いので、スマホ対策としてレスポンシブデザインが必要です。これは表示する画面の幅に応じて、あらかじめデザインを記述しておいて、CSS でデザインを自動で切り替えていく方法です。

CSS 中に @media を記述して、最大または最小の画面幅を指定して {} かっこの中にその場合のデザインを記述します。また HTML の head の中に

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

を記述しておきます。

【css/style.css の続き】

```
/* スマホ対応 */
@media (max-width: 600px) {
  .login-container {
    width: 100%; /* スマホのときの幅 */
    box-sizing: border-box; /* パディングとボーダーを含める */
    margin: 10px; /* 画面端から少し余裕を持たせる */
  }
  .login-container button {
    width: 98%;
  }
}
```

この場合 600px までの範囲ではこのデザインが当てはまり、それより大きい場合は()の外のデザインが適応されます。

できれば、このようなスマホ対応を先にデザインする方が良いでしょう。これをモバイルファーストといいます。Google Analytics などで見ると、参考に私個人のサイトでは 86%がスマホです。もっとも多い画面サイズは 390×844 でした。つまり iPhone の 13 以降ですね。

ハートマークなどの記号

Web ページでハートマークや×マークなどを表示するにはどうしたらいいでしょうか。アイコンを作る方法もありますが、font awesome という SVG を使う方法があります。これは有料と無料のサービスがありますが、簡単なものであれば無料で使うことができます。また CDN を使用することができるので、準備が簡単です。

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.7.2/css/all.min.css">
```

図 4-6 旧式の css 指定

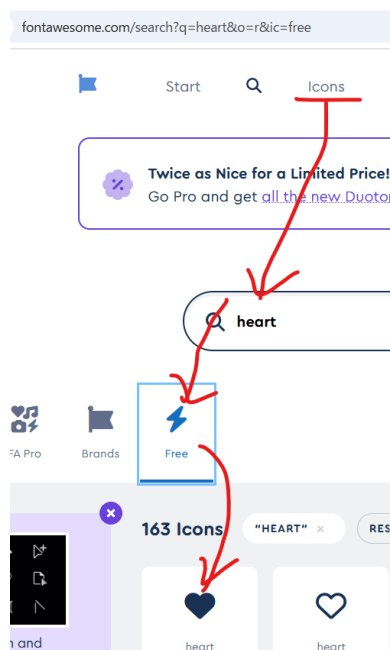
この方法でもできますが、正式ではありません。正式には

```
<script src="https://kit.fontawesome.com/*****.js"
crossorigin="anonymous"></script>
```

図 4-7 正しい css の指定 **部分を書き換え

Fontawesome.com でユーザ登録して「Download」の一番下にある「Use Font Awesome Kit」から「Add New Kit」で Free の Kit を作成してください。するとこれが CDN に登録されるので、10 桁の英数の js ファイルができますので、その名前を上のコードの*の部分と書き換えます。

たとえば、ハートマークを出すとしましょう。Fontawesome.com に行って、icons の中で heart を検索します。



さらに Free をクリックすると無料のアイコンだけ表示されるので、その中から適当な ICON を選択します。

図 4-8 font awesome での検索

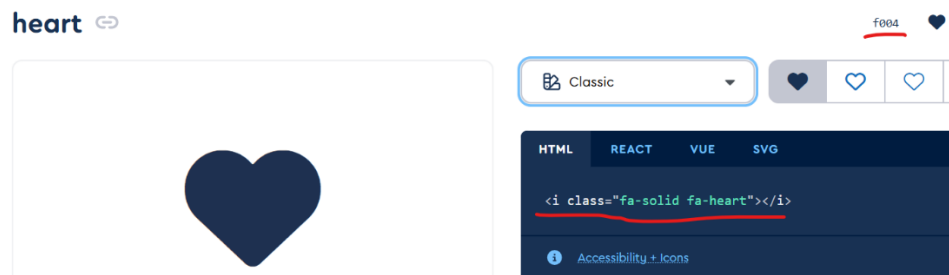


図 4-9 ハートの font

展開すると、その font の番号の f004 や、表示するための html が表示されるので、それを HTML 中に記述します。<i>でやる方式と css で font 番号を指定する方式です。

【fonttest.html 一部】

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.7.2/css/all.min.css">
<style>
  #batu::before {
    font-family: "Font Awesome 6 free";
    font-weight: 900;
    content: "\f057";
    color: red;
    padding-right: 10px;
  }
</style>
</head>
<body>
  <p>heart: <i class="fa-solid fa-heart"></i></p>
  <p id="batu">css font</p>
</body>
</html>
```

フォーム情報をチェックする

では、HTML も PHP の実験もできましたので、組み合わせていきましょう。

【join/index.php】

```
<?php
if (!empty($_POST)) {
    //ファイルタイプチェック
    $fileName = $_FILES['image']['name'];
    if (!empty($fileName)) {
        $ext = mb_strtolower(substr($fileName,-4));
        if ( $ext != '.jpg' && $ext != '.gif' && $ext != '.png') {
            $error['image'] = '未対応のファイルタイプです';
        }
    }
    if (empty($error)) {
        header('Location: check.php');
        exit();
    }
}
?>
<!DOCTYPE html>
. . .
```

Form で action が指定してないので、自分自身で POST で form データを受信することになります。最初の if 文が必要なのは、まだ form を入力していない最初の時と、form にデータが入力され submit された時の 2 種類の突入があるからです。1つ目の時は、なにもすることはありませんので、そのままスルーして form の入力に向かいますが、2つ目の突入では、入力されたデータをチェックする必要があります。

今回は、input に required 属性を付けているので入力なしで2つ目の突入になることはないので、とりあえずファイルタイプのチェックだけやってみましょう。

form の enctype に `enctype="multipart/form-data"` つけているので、ファイルの選択送信をすることができるようになっています。この場合は `type="file"` の name で命名された `$_FILES['image']` の中に、ファイル名、テンポラリーファイル名などが入っているのでここから情報をチェックします。実ファイル名の後ろ 4 文字を取得して、許可しているファイルタイプかチェックします。これらのファイルタイプでなければ、`$error['image']` という連想配列を作り、エラー理由を入れておきます。そして、そのエラーがなければ、次の確認画面に JUMP します。

エラーがあった場合は、JUMP しないでそのまま form 入力に戻りますが、その時ファイルタイプがエラーであることを表示したいですね。そこで次のように記述します。

【join/index.php の form 部分】

```
<input type="file" class="joinFile" name="image" size="120">
<?php if (isset($error['image'])): ?>
<span class="error"><?php echo $error['image']; ?></span>
<?php endif; ?>
```

If 文で\$error['image']が存在するか調べます。エラーがある時だけこの連想配列は存在しますので、あれば次の行以降を表示します。If 文の最後の:は、この後の行から endif;までが if 文の範囲であることを示しています。ここで span で配列の内容を表示しています。

【css/style.css の一部】

```
.error {
    color: red;
}
/* font awesome x-mark */
.error::before {
    font-family: "Font Awesome 6 free";
    font-weight: 900;
    content: "\f057";
}
```

また、目立つように×マークを css で表示します。CSS で awesome を使用した場合はこのように表示されます。Awesome を使用するときには図 4-7 のように awesome を指定してください。



図 4-10 エラー表示

さて続いてファイルが指定されている時には、ファイルを指定のフォルダーに移動させる処理があります。この時 AMPPS で実行するとファイル名に全角が使用されていると文字化けを起こして移動に失敗します。これは Windows の場合ファイル名が Shift-JIS で登録されているからです。PHP の内部では UTF-8 なので移動する関数でファイル名をそのまま指定すると文字が化けてしまいます。そこで Windows の時だけあえてファイル名を Shift-JIS に変更します。それ以外の OS ではそのままです。

【join/index.php の一部】

```
if (empty($error)) {  
    //画像を格納  
    $image = date('YmdHis').$fileName;  
    if (DIRECTORY_SEPARATOR == '¥¥') { //windows の場合  
        $imagename = mb_convert_encoding($image, "sjis", "utf8");  
    } else {  
        //それ以外  
        $imagename = $image;  
    }  
    move_uploaded_file($_FILES['image']['tmp_name'], '../member_image/'.$imagename);  
    header('Location: check.php');  
    exit();  
}
```

これでファイルが指定してあると、ファイル名の頭に日付時刻を付けたファイルが/member_image に UPLOAD されると思います。されていない場合は form の enctype の中のスベルか/member_image の 名前の間違いや場所の間違いを探してください。

ファイル名だけだと偶然ファイル名が同じになったときに区別できないので日付時刻を頭につけています。

セッション

Index.php の上部で入力値をチェックして OK であれば、そのデータを check.php に送る必要があります。check.php はそれを受けて入力項目を画面に表示して、入力者に確認を促します。

index.php で form に入力されたデータは form の POST によって自分自身に送信されて、\$_POST スーパーグローバル変数で受信することができました。しかし、この\$_POST は次に渡す 1 回しか使えません。index.php から check.php にはまた別の方法を使わなければ渡りません。

そこで今回は\$_SESSION スーパーグローバル変数を使用します。これを使うと同じセッション使用中はそのデータがサーバ側に保存されて、何回でも使用できます。セッションが切れると、使用できなくなります。この後 ID とパスワードを入力してログインに成功したという情報も保存できるわけです。

この\$_SESSION を使用するには、ページの先頭(<!DOCTYPE>より前)で session_start()関数を呼び出す必要があります。セッションは Cookie に ID で登録されます。そしてその ID を使って/tmp フォルダなどに関係するファイルが保存されます。この関係を関連づけてくれるのが session_start 関数です。なのでブラウザを閉じてセッション ID が変わってしまえば、前に使ったセッションデータにはもう結びつけられません。

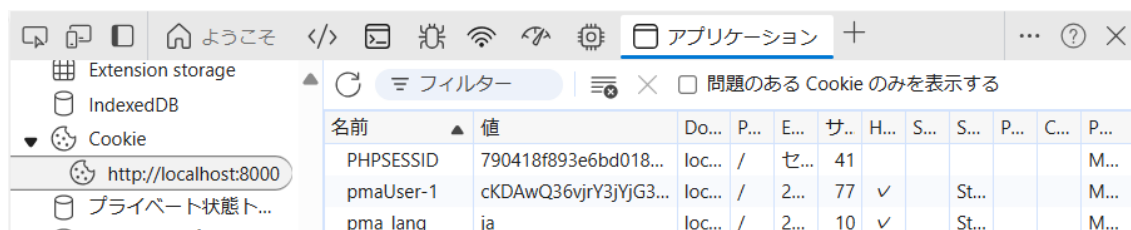


図 4-11 Cookie のセッション ID

```
docker compose exec apache /bin/bash
ls /tmp
sess_790418f893e6bd018458b039ea718cce
cat /tmp/sess*
join|a:4:{s:4:"name";s:10:"3525235235";s:4:"mail";s:13:"abc@for.co.jp";s:4:"pas
s";s:9:"sdgdsagag";s:5:"image";s:14:"20250131012328";}
```

図 4-12 tmp フォルダの中身

Cookie の PHPSESSID と同じ番号のファイルが/tmp フォルダにできており、その中身にセッションのデータが書かれています。

では、この部分のソースを確認しましょう。

【join/index.php 冒頭部分】

```
<?php
session_start();

if (!empty($_POST)) {
```

【join/index.php JUMP 部分】

```
move_uploaded_file($_FILES['image']['tmp_name'], '../member_image/'.$imagena
me);
$_SESSION['join'] = $_POST;
$_SESSION['join']['image'] = $image;
header('Location: check.php');
```

\$_SESSION に連想配列で join を作り、その中に\$_POST で渡ったものをそのまま代入します。ただファイル名は\$_FILES で渡って来ていますので、\$_SESSION に代入しますが、Shift-JIS に変更していない方のファイル名を代入します。

渡されたセッション情報を表示する

それでは、渡されたセッション情報を check.php で表示しましょう。check.php の先頭でも同じように session_start() を呼びます。

【join/check.php】

```
<?php
session_start();
?>
<!DOCTYPE html>
...
```

【join/check.php body 内部】

```
<body>
  <div class="check-container">
    <h1>登録確認画面</h1>
    <p>内容を確認してください</p>
    <form action="" method="post">
      <dl>
        <dt>ニックネーム</dt>
        <dd>
          <?php echo $_SESSION['join']['name']; ?>
        </dd>
        <dt>メールアドレス</dt>
        <dd>
          <?php echo $_SESSION['join']['mail']; ?>
        </dd>
        <dt>パスワード</dt>
        <dd>
          <?php echo $_SESSION['join']['pass']; ?>
        </dd>
        <dt>画像データ</dt>
        <dd>
          <?php echo $_SESSION['join']['image']; ?>
        </dd>
      </dl>
      <input type="hidden" name="mode" value="submit">
      <button type="submit">登録</button>
      <button type="button" class="cancel"
        onclick="location.href='index.php?mode=redo'">キャンセル</button>
    </form>
  </div>
</body>
```

【css/style.css】

```
.login-container button,
.check-container button {
  opacity: 0.8;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  background-color: #495aa5;
  color: white;
  cursor: pointer;
  width: 100%;
  margin-bottom: 3px;
}
.login-container button:hover,
.check-container button:hover {
  opacity: 1.0;
}
/* 確認画面 */
.check-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  width: 300px; /* デフォルトの幅 */
}
.check-container form {
  width: 100%;
}
.check-container dl {
  display: flex;
  flex-direction: column;
  align-items: start;
}
.check-container dt {
  width: 97%;
  padding: 3px 0 3px 5px;
  background-color: #d0dfee;
}
.check-container dd {
  width: 100%;
  padding: 3px 0 5px 0;
}
```

```
.check-container .cancel {
  opacity: 0.8;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  background-color: #dea412;
  color: white;
  cursor: pointer;
  width: 100%;
}
```

図 4-13 登録確認画面

まずは、渡したものがちゃんと渡せたか確認するために、そのまま表示しています。

しかし、パスワードはあまり表示すべきではありませんので表示をやめましょう。また画像ファイルは何も指定しなくても何か渡って来ています。これはファイル名の頭に付けた日付時刻の情報です。そこでここに関しては渡す側で未指定ならば「noimage.jpg」のようなファイル名にして、/member_image に未指定用の画像をあらかじめ入れておくことにしましょう。そしてファイル名ではなく、実際に渡された画像を小さく表示するように改造します。

Input type="hidden"が一つ入っています。実はこれがないと、form が飛ばす内容が無いので POST できません。ダミーデータのようなものです。

【css/style.css @media 以降変更】

```
/* スマホ対応 */
@media (max-width: 600px) {
  .board-container,
  .check-container,
  .thanks-container {
    width: 98%;
    box-sizing: border-box;
    margin: 10px;
  }
}
```

スマホ対応の CSS を付け加えておきます。

【join/index.php 改造】

```
//画像を格納
$image = date('YmdHis').$fileName;
if ($_FILES['image']['name']=='') {
    $image = 'noimage.jpg';
} else {
    if (DIRECTORY_SEPARATOR == '¥¥') { //windows の場合
        $imagename = mb_convert_encoding($image,"sjis","utf8");
    } else {
        //それ以外
        $imagename = $image;
    }
    move_uploaded_file($_FILES['image']['tmp_name'],'../member_image/'.$image
name);
}
```

【join/check.php 改造部分】

```
<dt>パスワード</dt>
<dd>
    表示しません
</dd>
<dt>画像データ</dt>
<dd>
    
</dd>
```

【css/style.css】

```
.check-container img {
    width: 100px;
    padding: 2px;
    border: #555 solid 1px;
}
```

登録確認画面

内容を確認してください

ニックネーム

3525235235

メールアドレス

abc@for.co.jp

パスワード

表示しません

画像データ



登録

キャンセル

図 4-15 画像未指定

登録確認画面

内容を確認してください

ニックネーム

3525235235

メールアドレス

abc@for.co.jp

パスワード

表示しません

画像データ



登録

キャンセル

図 4-14 画像指定

だいぶ整ってきました。check.php で確認ができれば「登録」を押してデータベースに登録です。これは check.php の冒頭部分で行います。

共通ルーチン

データベースに接続する部分を共通ルーチンとして外部に出しましょう。こうすることで join の内部やそれ以外の部分でデータベースに接続する場合のルーチンを共通化できます。Testuser,testpass はユーザ名とパスワードです。phpMyAdmin で設定した root などのユーザ名と、設定したパスワードを指定してください。指定しなければ testpass の部分は''にして空っぽにしてください。このファイルは/join フォルダーの親フォルダーに作成します。

【dbconnect.php】

```
<?php
try {
    $dsn = 'mysql:host=db;dbname=bbs;charset=utf8';
    $db = new PDO($dsn, 'testuser', 'testpass');
    // echo "接続に成功しました";
} catch (PDOException $e) {
    echo "接続に失敗しました";
    echo $e->getMessage();
    exit();
}
?>
```

上は実験で行った接続部分を切り取って dbconnect.php に出力したものです。そして check.php からこの dbconnect.php を呼び出すためにソースの変更が必要です。ここではまだ関係ないのですが、このあと fetchAPI を使う関係で余分な出力はエラーになるので成功時の echo のメッセージは止めておきます。

【join/check.php】

```
<?php
session_start();
require('../dbconnect.php');
?>
<!DOCTYPE html>
<html lang="ja">
```

dbconnect.php を require 関数で呼び出すと、それ以降\$db 変数に PDO クラスのオブジェクトができるので、データベースと接続できるようになります。

ではついでにデータベースアクセスするときによく使う htmlspecialchars()関数のラッパーをこの中に作っておきましょう。htmlspecialchars 関数は、form などでユーザ入力された文字を表示する際に使うもので、ユーザが悪意で javascript を入力してもそれが実行されないようにする機能を持っています。

たとえば掲示板で

```
<script>location.href="https://www.yahoo.co.jp";</script>
```

という文字列を入力したとしましょう。するとこれがデータベースに登録されて、次の表示の時に、この文字列が表示されますが、この文字列は JavaScript なので表示されたと同時に、実行されてしまいます。これが実行されると yahoo の画面に切り替わってしまいます。つまりもう、その掲示板が見れなくなってしまいます。

このように Web では、ユーザの入力したものをそのまま表示すると色々な不都合が発生します。この関数は表示するときにコメントとして表示するようにしてくれますので、実行されずただ表示されるだけになります。ただこの関数の名前が長いので面倒です。そこで省略で呼べるように共通関数に function を登録しておきましょう。

【dbconnect.php に追加】

```
//htmlspecialchars のショートカット  
function h($value) {  
    return htmlspecialchars($value, ENT_QUOTES);  
}
```

Index.php と check.php で htmlspecialchars を記述すべきところを h() に書き換えて記述します。

【join/check.php h() の追加】

```
<dl>  
    <dt>ニックネーム</dt>  
    <dd>  
        <?php echo h($_SESSION['join']['name']); ?>  
    </dd>  
    <dt>メールアドレス</dt>  
    <dd>  
        <?php echo h($_SESSION['join']['mail']); ?>  
    </dd>  
    <dt>パスワード</dt>  
    <dd>  
        表示しません  
    </dd>  
    <dt>画像データ</dt>  
    <dd>  
          
    </dd>  
</dl>
```

ユーザ登録

index.php で入力し、check.php で確認したデータをデータベースに登録しましょう。登録するべきデータは、セッションに入っています。

【join/check.php】

```
<?php
require('../dbconnect.php');
session_start();

//index.php 経由でない場合は登録に戻る
if (!isset($_SESSION['join'])) {
    header('Location: index.php');
    exit();
}
if (!empty($_POST)) {
    $mname = $_SESSION['join']['name'];
    $email = $_SESSION['join']['mail'];
    $motopass = $_SESSION['join']['pass'];
    $pass = password_hash($motopass,PASSWORD_DEFAULT);
    $image = $_SESSION['join']['image'];
    try {
        $statement = $db->prepare('insert into members
(mname,email,pass,picture,created) values(?,?,?,?,now());');
        $statement->bindParam(1, $mname,PDO::PARAM_STR);
        $statement->bindParam(2, $email,PDO::PARAM_STR);
        $statement->bindParam(3, $pass,PDO::PARAM_STR);
        $statement->bindParam(4, $image,PDO::PARAM_STR);
        $statement->execute();
    } catch(PDOException $e) {
        $error['insert'] = $e->getMessage();
    }
    if (empty($error)) {
        header('Location: thanks.php');
        exit();
    }
}
?>
<!DOCTYPE html>
```

これでデータベースへの登録ができました。

最初に `isset($_SESSION['join'])` を見っていますが、これは index.php で入力して来たのではなく、url を直接入力して check.php に来た場合の対応です。これを許してしまうと適当なデータで勝手に登録されてしまいます。またエラーが発生した場合はエラーの原因を表示するようにしておきます。

しかし、まだ調整すべき点が残っています。

確認でキャンセルした場合

check.php で確認した結果正しくなかった場合に「キャンセル」ができるようにしていますが、index.php のエラーの時と同様に入力した値が再表示されません。このあたりを調整しておきましょう。またこの後データベースも使うので dbconnect.php を読み込んでおきます。

【join/index.php の冒頭】

```
<?php
session_start();
require('../dbconnect.php');
```

【join/index.php DOCTYPE の直前】

```
//再入力の場合
if (isset($_GET['mode']) && $_GET['mode'] == 'redo') {
    $_POST = $_SESSION['join'];
    $error['redo'] = true;
}
?>
<!DOCTYPE html>
```

【join/index.php form の中】

```
<form action="" method="post" enctype="multipart/form-data">
    <input type="text" placeholder="Nick Name" name="name" size="35"
minlength="4" maxlength="100" value="<?php echo h($_POST['name']) ?? ''; ?>"
required>
    <input type="email" placeholder="Mail Address" name="mail"
size="80" value="<?php echo h($_POST['mail']) ?? ''; ?>" required>
    <input type="password" placeholder="Password" name="pass" size="20"
minlength="8" maxlength="20" value="<?php echo h($_POST['pass']) ?? ''; ?>"
required>
    <input type="file" class="joinFile" name="image" size="120">
    <?php if (isset($error['image'])): ?>
    <span class="error"><?php echo $error['image']; ?></span>
    <?php endif; ?>
    <?php if (isset($error['redo'])): ?>
    <span class="warning">画像を改めて選択してください</span>
    <?php endif; ?>
```

【css/style.css 一部】

```
.warning {
    color: #dea412;
}
/* font awesome exclamation */
.warning::before {
    font-family: "Font Awesome 6 free";
    font-weight: 900;
    content: "✖️";
    padding-right: 10px;
}
```

?? は null 合体演算子で、null の時に何に変換するかを記述します。\$_POST が無いときに h(\$_POST['pass']) とすると、そのオブジェクトが null なのでワーニングになってしまいます。そこで h(\$_POST['pass'] ?? '') のようにすると null の場合には空文字列に置き換えてくれるのでワーニングになりません。

登録済みのユーザの時

members テーブルは、mid が主キーですから mail が同じものが登録されてもエラーにはなりません。しかし LOGIN では mail を使用するのでこれでは困ります。そこで mail の重複チェックを追加したいと思います。

【join/index.php if (empty(\$error))の直前】

```
//重複アカウントのチェック
$member = $db->prepare('SELECT COUNT(*) as cnt FROM members WHERE email=?;');
$member->execute(array($_POST['mail']));
$record = $member->fetch();
if ($record['cnt'] > 0) {
    $error['mail'] = '登録済みのメールアドレスです';
}

if (empty($error)) {
```

bindParam 以外にも、execute に対して array で引数を記述すれば prepare を実行できます。引数が1つ以上の場合は array の中に、複数個代入します。Execute の引数は array 一つしか指定できません。

登録確認画面

登録ができれば、登録確認画面を表示します。確認のためにニックネームを表示するので\$_SESSION を使います。そして LOGIN で再度 ID,password を入力してもらうために SESSION を削除します。後は LOGIN 画面に移動します。

【join/thanks.php】

```
<?php
session_start();
//index.php 経由でない場合は登録に戻る
if (!isset($_SESSION['join'])) {
    header('Location: index.php');
    exit();
}
//ニックネーム取得
$nick = $_SESSION['join']['name'];
//初期化
unset($_SESSION['join']);
?>
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>登録完了画面</title>
    <link rel="stylesheet" href="../css/style.css">
</head>
<body>
    <div class="thanks-container">
        <h1>登録完了画面</h1>
        <p><?php echo $nick; ?>さんを登録しました</p>
        <button type="button" class="submit"
            onclick="location.href='../'">LOGIN へ</button>
    </div>
</body>
</html>
```

LOGIN サブシステム

ユーザ登録ができたので、今度は認証です
さきほど HASH で保存した内容と手入力の値の比較で認証を行います

5. LOGIN サブシステム

あらかじめ試しておきたい処理

■ SELECT 文実行

ログインするには認証が必要になります。認証するには前の章でやったパスワードを読み込む必要があります。いままではデータベースに対する接続と挿入を実行しましたが、読み込みの時には少し違った処理が必要になります。

INSERT、DELETE、UPDATE などは、実行したら成功したか失敗したかの結果を取得しますが、それ以外に返ってくるものはありません。しかし SELECT の場合は合致したレコードの内容を受信しなければなりません。そのため少し違ったプログラムになります。

【login.php 省略あり】

```
<?php
require('dbconnect.php');
?>
.....
<body>
<?php
$mid = 5;
try {
    $statement = $db->prepare('select * from members where mid=?;');
    $statement->bindParam(1, $mid, PDO::PARAM_INT);
    $statement->execute();
    $record = $statement->fetch(PDO::FETCH_BOTH);
    echo "SELECT 成功";
} catch(PDOException $e) {
    echo "SELECT 失敗";
    exit();
}
echo sprintf("<br>mid=%d mname=%s
pass=%s", $record['mid'], $record['mname'], $record['pass']);

$statement = null;
$db = null;
?>
</body>
```

今回は SELECT で mid を指定して 1 レコードだけ読み込む実験です。やはり prepare を使って指定しています。fetch を行うことで合致したレコードの先頭の 1 レコードだけ読みこまれます。

fetch でのデータ取得にはいくつか種類があって、なにも指定していない場合は、FETCH_BOTH となり、配列とカラム名の 2 種類返ってきます。

option	返還方法	例
FETCH_BOTH(デフォルト)	カラム名+配列	\$rec['mid'] / \$rec[0]
FETCH_ASSOC	カラム名	\$rec['mess']
FETCH_KEY_PAIR	キーと値の2つ	\$rec[3] キーを指定
FETCH_COLUMN	配列	\$rec[0] 配列インデックスを指定

図 5-1 FETCH オプション

■ 認証方法

さて、HASH 化のところでも述べましたが、PASS は HASH 化して保存してあります。なのでユーザが入力したパスワードとは合致しません。password_hash()に保存したパスワードは password_verify()で認証ができます。

【login.php 追加分】

```

echo sprintf("<br>mid=%d mname=%s
pass=%s",$record[ 'mid' ],$record[ 'mname' ],$record[ 'pass' ]);

if (password_verify("12345678",$record[ 'pass' ])) {
    echo "<br>認証 OK";
} else {
    echo "<br>認証失敗";
}

?>
</body>

```

Password が 12345678 であれば通ります。違う Password であればソースを修正してください。

LOGIN 画面

実験ができたので、本番画面を作成しましょう。

【login.php】

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.7.2/css/all.min.css">
</head>
<body>
  <div class="login-container">
    <h1>ログイン画面</h1>
    <p><a href="join/index.php">ユーザでない方は登録へ</a></p>
    <form action="" method="post">
      <input type="text" placeholder="Mail Address" name="mail" size="100"
required>
      <input type="password" placeholder="Password" name="pass" size="20"
required>
      <button type="submit">Login</button>
    </form>
  </div>
</body>
</html>
```

ユーザ登録と同じ流れで作っていきます。css も共通にします。そのほかの画面でも共通にしたいところもあるので一度 CSS のまとめを見ましょう。

【css/style.css 変更分】

```
/* Login 用 */
.login-container,
.check-container,
.thanks-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    ...

.login-container button,
.check-container button,
.thanks-container button {
    opacity: 0.8;
    padding: 10px 20px;
    border: none;
    ...

.login-container button:hover,
.check-container button:hover,
.thanks-container button:hover {
    opacity: 1.0;
}
...

/* スマホ対応 */
@media (max-width: 600px) {
    .board-container,
    .check-container,
    .thanks-container {
        width: 98%;
        box-sizing: border-box;
        margin: 10px;
    }
}
```

これでまず form ができました。入力して SUBMIT すると login.php に POST データが渡りますので、\$_POST がある場合はチェックしてパスワードの認証を行いましょう。

【login.php 冒頭部】

```
<?php
require('dbconnect.php');
session_start();

if (!empty($_POST)) {
    $mail = h($_POST['mail']);
    $pass = h($_POST['pass']);
    try {
        if ($mail != '' && $pass != '') {
            $statement = $db->prepare('select * from members where email=?;');
            $statement->bindParam(1, $mail,PDO::PARAM_STR);
            $statement->execute();
            $record = $statement->fetch(PDO::FETCH_BOTH);
        }
        if (!empty($record)) {
            if (password_verify($pass,$record['pass'])) {
                $_SESSION['id'] = $record['mid'];
                $_SESSION['time'] = time();

                header('Location: index.php');
                exit();
            } else {
                $error['login'] = "認証できませんでした";
            }
        } else {
            $error['login'] = "認証できませんでした";
        }
    } catch(PDOException $e) {
        $error['access'] = "アクセスできませんでした";
    }
}
?>
<!DOCTYPE html>
```

データベースへのアクセスでエラーになった場合は「アクセスできませんでした」のエラー、認証のエラーの場合は「認証できませんでした」と表示したいので、その部分を追加します。また認証エラーの場合に、メールアドレス、パスワードをまっさらにして再入力では大変なので、入力値を再表示するようにします。ただし、初回の場合は\$mail 等がありませんので、NULL 合同演算子を用いてワーニングが出ないように処理します。

```
<form action="" method="post">
  <input type="text" placeholder="Mail Address" name="mail" size="100"
value="<?php echo $mail ?? ''; ?>" required>
  <input type="password" placeholder="Password" name="pass" size="20"
value="<?php echo $pass ?? ''; ?>" required>
  <?php if (isset($error['access'])): ?>
  <span class="error"><?php echo $error['access']; ?></span>
  <?php endif; ?>
  <?php if (isset($error['login'])): ?>
  <span class="error"><?php echo $error['login']; ?></span>
  <?php endif; ?>
  <button type="submit">ログイン</button>
```

クッキー (Cookie) を用いて、ID、PASSWORD の入力を省略する方法がありますが、Cookie の利用は色々制限が多くなっているので、今回は行いません。

投稿画面

ログインができたらいよいよメインの機能の投稿画面の作成です
投稿するだけでなく、コメントの入力や表示、削除などが必要になります

6. 投稿画面

あらかじめ試しておきたい処理

■ メッセージ書き込み

メッセージの書き込みは他の画面と同様に行いますが、メッセージは messages テーブルに行います。
\$id は適当に変更してください。

【databasetest.php 変更部分のみ】

```
<?php
require('dbconnect.php');
?>
.....
<body>
<?php
$message = '最初のテストデータです';
$id = 5;
try {
    $statement = $db->prepare('insert into messages (message,mid,mecre,deleted)
values(?,?,now(),false);');
    $statement->bindParam(1, $message,PDO::PARAM_STR);
    $statement->bindParam(2, $id,PDO::PARAM_INT);
    $statement->execute();
    echo "Insert 成功";
} catch(PDOException $e) {
    echo "Insert 失敗";
}
$statement = null;
$db = null;
?>
</body>
```

■ いいねのための Fetch API

メッセージの書き込みの場合は、form から submit することによって、index.php の上部でデータの受け取りとデータベースへの書き込みを行うので、そのあとデータベースの追加された内容を読み直して再表示します。しかし、いいねの場合はいいねを押した瞬間にいいねがデータベースに追加され、それによっていいねの件数が増えたことをその場出力さなければなりません。似たような実験ソースを作り、実験してみましょう。

Fetch API という JavaScript の標準 API を使用した方法を使います。以前は JQuery を使って、Ajax という方法で行われていましたが、最近 JQuery が使われなくなって来ています。Remix や Next.js の App router などありますが、標準だけでできるものにします。

【fetchsample.php】

```
...
<script>
  async function postData( argURL , argData ) {
    try {
      const response = await fetch(
        argURL , {
          method:'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify( argData ),
        }
      ) ;
      if ( !response.ok ) {
        throw 'Fetch error. HTTP Status:' + response.status ;
      }
      const resData = await response.json() ;
      return {
        status: true ,
        retData: resData ,
      } ;
    } catch( e ) {
      return {
        status: false ,
        retData: e ,
      } ;
    }
  }
}
```

```

    async function getProfile( argName, outTag ) {
        const data = { name : argName } ;
        try {
            const res = await postData( 'serverData.php', data ) ;
            if ( !res.status ) {
                throw res.retData ;
            }
            const retData = res.retData.data ;
            if ( res.retData.status ) {
                const profile = `${retData.area} ${retData.taste}` ;    // プ
                var outdiv = document.getElementById(outTag);
                outdiv.innerHTML = profile;
                console.log(profile);
            } else {
                console.error( retData.reason ) ;
            }
        } catch( e ) {
            console.error(e) ;
        }
    }
    async function coffeeProfile() {
        await getProfile( 'KILIMANJARO', 'name1' ) ;
        await getProfile( 'Blue Mountain', 'name2' ) ;
        await getProfile( 'Mocha', 'name3' ) ;
        await getProfile( 'Japan', 'name4' ) ;
    }
</script>
</body>

```

プロフィールの編集

JavaScript もソースに入れました。続いて実験用の HTML です。

```
</head>
<body>
  <button onclick="coffeeProfile();">呼び出し</button>
  <div id="name1"></div>
  <div id="name2"></div>
  <div id="name3"></div>
  <div id="name4"></div>
  ...
</body>
</html>
```

```
@charset "utf-8";

body {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
  font-family: Arial, sans-serif;
  min-width: 375px;
}
```

これが HTML と JavaScript 呼び出し部分で、「呼び出し」ボタンを押すと、coffeeProfile 関数が呼び出されて、検索用の data とどこの id の div に表示するのかが指示されます。そこから postData 関数で fetch を使って、指定した PHP ファイルが呼び出されます。

【serverData.php】

```
<?php
$profileAry = array(
    'KILIMANJARO' => array(
        'area' => 'Tanzania' ,
        'taste' => 'Strong acidity and richness' ,
    ) ,
    'BLUE MOUNTAIN' => array(
        'area' => 'Jamaica' ,
        'taste' => 'Harmonious taste' ,
    ) ,
    'MOCHA' => array(
        'area' => 'Tanzania' ,
        'taste' => 'Fruity sour and sweet' ,
    ) ,
    'GUATEMALA' => array(
        'area' => 'Guatemala' ,
        'taste' => 'Fresh acidity reminiscent of fruit' ,
    ) ,
);
$retAry = array() ; // 返信用配列データ
$req = json_decode( file_get_contents( 'php://input' ), true ); // リクエスト
// データを取得 *注1
$reqName = strtoupper( $req[ 'name' ] );
if ( array_key_exists( $reqName , $profileAry ) ) {
    $retAry[ 'status' ] = true ;
    $retAry[ 'data' ] = $profileAry[ $reqName ] ;
} else {
    $retAry[ 'status' ] = false ;
    $retAry[ 'data' ][ 'reason' ] = $req['name'] . ' is not found!' ;
}
echo( json_encode( $retAry ) ); // JSON 形式でデータを返しま
// す。
?>
```

ここからは PHP ファイルの説明です。Fetch で起動された serverData.php(同じフォルダーにある)が、リクエストの文字列を php://input から受け取り、検索用の文字を name 要素から受信して、PHP が持っている配列から検索して、結果を status と data に詰めて、また json 型で送り返します。

ここから JavaScript の postData の await fetch に戻って json データを受信します。

続いて return することで getProfile()に戻り、指定された id の場所を探して、内容を innerHTML で表示します。getProfile の呼び出しを 4 回やっていますから、これを 4 回繰り返しています。

coffeeProfile->getProfile->postData->serverData.php

div 出力<-getProfile<-postData<-

今回は、配列からの検索でしたが、これをデータベース接続して INSERT した結果を SELECT で検索するように変更すれば、いいねの fetch API の出来上がりになります

■ モーダル画面

モーダル画面とは、その画面で OK や CANCEL をしない限り、その下に隠れている画面が触れないようにした画面です。今回の例で言えば、各メッセージに対してコメントを入力する場面で使いたいです。ではまた実験をしてみましょう。テストなので awesome font は使いません。

【modaltest.php】

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>モーダル実験</title>
  <link rel="stylesheet" href="css/style.css">
  <script>
    function reply_open(id) {
      var rep = document.getElementById("post"+id);
      rep.style.display = rep.style.display == "block" ? "none" : "flow";
      var parent = rep.parentNode;
      parent.style.display = parent.style.display == "block" ? "none" :
"flow";
    }
  </script>
</head>
```

コメントボタンを押したら、display の block と flow を切り替える reply_open 関数を呼び出します。この際、どの form を表示するのか分かるようにメッセージの ID を渡すようにします。

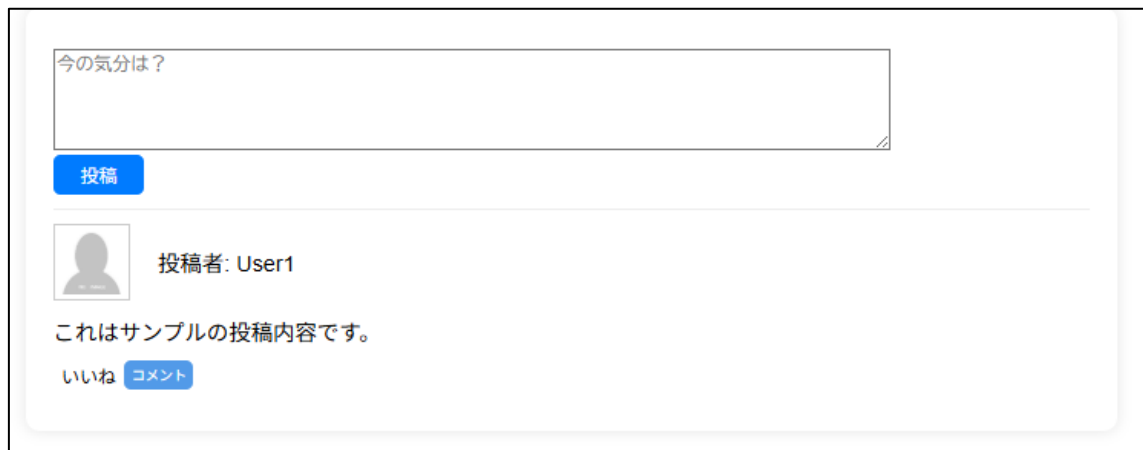
```

<body>
  <div class="board-container">
    <div class="mess">
      <form action="" method="post">
        <textarea name="message" rows="5" placeholder="今の気分は？"
"></textarea>
        <div class="mess-actions">
          <button type="submit">投稿</button>
        </div>
      </form>
    </div>
    <div class="post">
      <div class="post-header">
        
        <span>投稿者: User1</span>
      </div>
      <div class="post-content">
        これはサンプルの投稿内容です。
      </div>
      <div class="post-actions">
        <button class="pushLike">いいね</button>
        <button class="pushComment" onclick="reply_open(4);">コメント
</button>
        <div class="modal">
          <div class="replies" id="post4">
            <form action="" method="post">
              <textarea name="reply" rows="10" placeholder="コメント
">中身のサンプル</textarea>
              <button type="submit">書き込み</button>
              <button onclick="reply_open(4);return false;">キャンセ
ル</button>
              <input type="hidden" name="4">
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>

```

replies クラスの div にメッセージ ID が id 番号で振ってあります。例では 4 になっています。その配下の form に textarea が作っており、hidden で 4 という name が振られています。Submit された場合は textarea のデータと hidden のデータが POST で送出されます。またキャンセルボタンが押された場合は、form の表示を none にするように関数が呼ばれます。

通常表示時はコメントの form が none になっているのでこのように表示されます。



今の気分は？

投稿

 投稿者: User1

これはサンプルの投稿内容です。

いいね コメント

図 6-1 コメント入力前

コメントボタンを押すと form が表示されて、post4 の親である modal クラスが表示されます。この modal クラスは灰色の画面を全面に表示する CSS になっています。



今の気分は？

投稿

中身のサンプル

書き込み キャンセル

図 6-2 コメント入力中

【css/style.css】

```
/*
  掲示板用
*/
.board-container {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: flex-start;
  width: 80%;
  max-width: 800px;
  background-color: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
/* 投稿 */
.mess {
  display: flex;
  flex-direction: column;
  width: 100%;
  border-bottom: 1px solid #eee;
  padding: 10px 0 10px 0 ;
}
.mess textarea {
  width: 80%;
}
.mess-actions button{
  padding: 5px 20px;
  border: none;
  border-radius: 5px;
  background-color: #007BFF;
  color: white;
  cursor: pointer;
}
```



```

/* 投稿メッセージ表示 */
.post {
  display: flex;
  flex-direction: column;
  width: 100%;
  border-bottom: 1px solid #ccc;
  padding: 10px 0;
}
.post:last-child {
  border-bottom: none;
}
.post-header {
  display: flex;
  justify-content: space-start;
  align-items: center;
}
.post-header img {
  border: 1px solid #ccc;
  padding: 3px;
}
.post-header span {
  padding-left: 20px;
}
.post-content {
  margin: 10px 0;
}
/* いいね、コメント */
.post-actions {
  display: flex;
  justify-content: flex-start;
  align-items: center;
}
.post-actions .pushComment {
  margin-right: 10px;
  padding: 3px 5px;
  border: none;
  font-size: xx-small;
  border-radius: 5px;
  background-color: #529ae8;
  color: white;
  cursor: pointer;
}
.post-actions .pushLike {
  border: none;
  background-color: white;
  cursor: pointer;
}

```

```

/* 返信 */
.replys {
  display: none;
  position: fixed;
  z-index: 15;
  top: 30%;
  left: 50%;
  width: 80%;
  padding: 10px;
  background-color: #888;
  border-radius: 8px;
  -webkit-transform: translate(-50%, -10%);
  transform: translate(-50%, -10%);
  color: #fff;
  font-size: 1.3rem;
  border: 0.3rem solid #fff;
}
.replys textarea {
  width: 97%;
  border: 1px solid #42d261;
}
.replys button{
  padding: 0 5px;
  border: none;
  border-radius: 5px;
  background-color: #007bff;
  color: white;
  cursor: pointer;
}
.modal {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  background-color: rgba(152, 152, 152, 0.7);
  width: 100%;
  height: 100%;
  z-index: 10;
}

```

投稿フォームの設計

メッセージ書き込み、いいね、モーダルの実験ができたので、全部を組み合わせる投稿フォームの設計をしていきましょう。

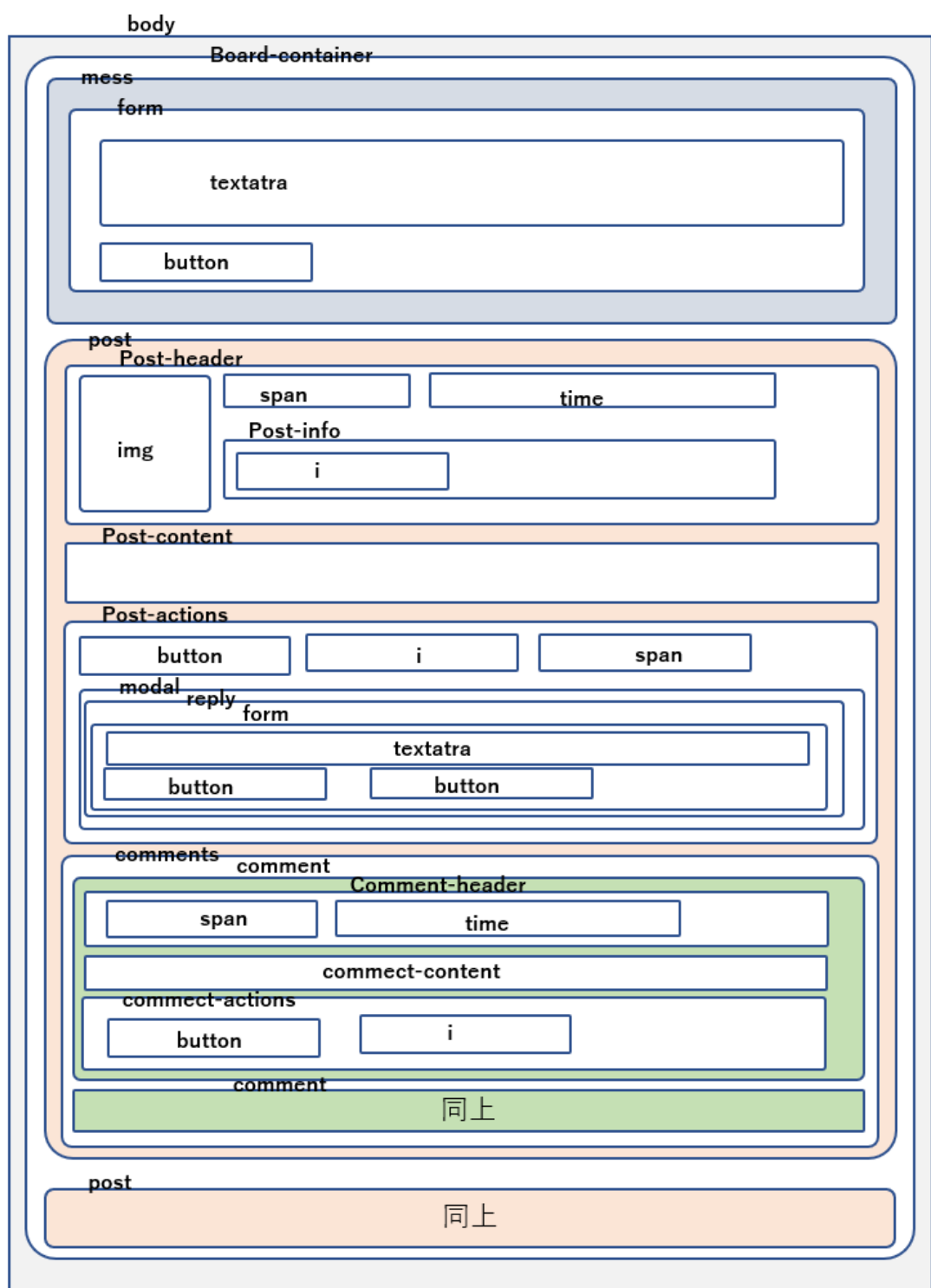


図 6-3 投稿画面設計

Board-container の中に「mess」「post」があり、post はメッセージ回数分繰り返します。Post の中は「post-header」「post-content」「post-actions」「comments」に分かれており、comments の中にコメントの数だけ comment が繰り返します。

Comment の中は「comment-header」「comment-content」「comment-actions」に分かれています。

このように階層を作っておき、データベースから読みこんだ内容を出力します。

まずは、プログラムではなく決め打ちの HTML と CSS でサンプルの投稿画面を作ってみます。
sampleindex.html です。modaltest.php などの応用です。

【sampleindex.html】

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>掲示板</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/6.7.2/css/all.min.css">
</head>
<body>
  <div class="board-container">
    <div class="mess">
      <form action="" method="post">
        <textarea name="message" rows="5" placeholder="今の気分は？"></textarea>
        <div class="mess-actions">
          <button type="submit">投稿</button>
        </div>
      </form>
    </div>
    <div class="post">
      <div class="post-header">
        
        <span>投稿者: User1</span>
        <span><time datetime="2024-12-23">2024-12-23</time></span>
        <div class="post-info">
          <span>
            <button class="pushDelete"><i class="fa-solid fa-trash warning"></i></button>
          </span>
        </div>
      </div>
      <div class="post-content">
        これはサンプルの投稿内容です。
      </div>
      <div class="post-actions">
        <button class="pushLike">
          <i class="fa-solid fa-heart fa-red"></i></button><span class="like-count">10</span>
        <button class="pushComment" onclick="reply_open(4);">コメント
      </div>
    </div>
  </div>
```

```

        <div class="modal">
            <div class="replies" id="post4">
                <form action="" method="post">
                    <textarea name="reply" rows="10" placeholder="コメント
">中身のサンプル</textarea>
                    <button type="submit">書き込み</button>
                    <button onclick="reply_open(4);return false;">キャンセ
ル</button>
                    <input type="hidden" name="4">
                </form>
            </div>
        </div>
    </div>
    <div class="comments">
        <div class="comment">
            <div class="comment-header">
                <span>コメント者: User2</span>
                <span><time datetime="2024-12-23">2024-12-
23</time></span>
            </div>
            <div class="comment-content">
                これはサンプルのコメント内容です。
            </div>
            <div class="comment-actions">
                <button class="pushComLike">
                    <i class="fa-solid fa-heart fa-red"></i></button>
            </div>
        </div>
        <div class="comment">
            <div class="comment-header">
                <span>コメント者: User3</span>
                <span><time datetime="2024-12-23">2024-12-
23</time></span>
            </div>
            <div class="comment-content">
                これはサンプルのコメント内容です。
            </div>
            <div class="comment-actions">
                <button class="pushComLike">
                    <i class="fa-regular fa-heart"></i></button>
            </div>
        </div>
    </div>
</div>

```

```

<div class="post">
  <div class="post-header">
    
    <span>投稿者: User2</span>
    <span><time datetime="2024-12-22">2024-12-22</time></span>
  </div>
  <div class="post-content">
    これはサンプルの投稿内容です。
  </div>
  <div class="post-actions">
    <button class="pushLike">
      <i class="fa-solid fa-heart fa-red"></i></button>
    <button class="pushComment" onclick="reply_open(3);">コメント
  </button>

  <div class="modal">
    <div class="replies" id="post3">
      <form action="index.php" method="post">
        <textarea name="reply" rows="10" placeholder="コメント
を入力してください"></textarea>
        <button type="submit">書き込み</button>
        <button onclick="reply_open(3);return false;">キャンセ
ル</button>

        <input type="hidden" name="hid" value="3">
      </form>
    </div>
  </div>
</div>
<div class="comments">
  <div class="comment">
    <div class="comment-header">
      <span>コメント者: User3</span>
      <span><time datetime="2024-12-22">2024-12-
22</time></span>
    </div>
    <div class="comment-content">
      これはサンプルのコメント内容です。
    </div>

    <div class="comment-actions">
      <button class="pushComLike">
        <i class="fa-regular fa-heart"></i></button>
      </div>
    </div>
  </div>
</div>

```

```

<script>
    function reply_open(id) {
        var rep = document.getElementById("post"+id);
        rep.style.display = rep.style.display == "block" ? "none" : "flow";
        var parent = rep.parentNode;
        parent.style.display = parent.style.display == "block" ? "none" :
"flow";
    }
</script>
</body>
</html>

```

掲示板関係の CSS を記述します。

【css/style.css time、削除、いいね部分】

```

time {
    font-style: italic;
    font-size: 0.8em;
}

/* 削除 */
.post-info {
    font-size: .75em;
    text-align: right;
    margin-left: 10px;
}
.post-info .pushDelete {
    border: none;
    background-color: white;
    cursor: pointer;
}
.post-info span {
    margin: 0 10px 0 0;
}
.post-info i {
    padding: 0 5px 0 10px;
}
.like-count {
    margin: 0 5px 0 10px;
}
.fa-red {
    color: red;
}

```


【css/style.css コメント部分】

```
/* コメント表示 */
.comments {
  display: flex;
  flex-direction: column;
  margin-top: 10px;
  font-size: small;
  border-left: 5px solid #ccc;
}
.comment {
  display: flex;
  flex-direction: column;
  padding: 5px 0;
  border-top: 1px solid #eee;
  margin-left: 20px; /* インデント */
}
.comment-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.comment-content {
  margin: 5px 0;
}
.comment-actions {
  display: flex;
  justify-content: flex-start;
  align-items: center;
}
.comment-actions .pushComLike {
  border: none;
  background-color: white;
  cursor: pointer;
}
```

.pushDelete は button の属性の中にアイコンを入れているため、ボタンが光らないように抑制しています。

削除ボタンの代わりにゴミ箱マークを入れました。またコメントのいいねはログインしたユーザーがいいねを入れたかどうかだけをハートの色で表示します。

今の気分は？

投稿

投稿者: User1 2024-12-23

これはサンプルの投稿内容です。

10

コメント

コメント者: User2

これはサンプルのコメント内容です。

コメント者: User3

これはサンプルのコメント内容です。

投稿者: User2 2024-12-22

これはサンプルの投稿内容です。

コメント

コメント者: User3

これはサンプルのコメント内容です。

図 6-4 投稿フォームのサンプル

コメントボタンでモーダルになります。いいねはデータベースが必要なのでまだ対応していません。

データー一覧

いままで色々実験をしてきたのでデータベースの内容がそれぞれになってしまいましたので、一度テーブルを消して、SQL を実行して投稿の一覧が表示できるようなデータを作ってみましょう。

まずは、メンバー表を消します。PhpMyAdmin などコマンドを実行しましょう。

```
delete from members;  
Alter table members auto_increment = 1;
```

図 6-5 メンバー表の初期化

続いてメッセージデータを消します。

```
delete from messages;

Alter table messages auto_increment = 1;
```

図 6-6 メッセージデータ初期化

メンバー表は HASH が必要なのでプログラムで実行します。

【databasetest.php ユーザ1の部分】

```
//user1
$name = 'user1';
$email = 'user1@abc.com';
$motopass = '12345678';
$pass = password_hash($motopass,PASSWORD_DEFAULT);
$image = 'noimage.jpg';
try {
    $statement = $db->prepare('insert into members
(mname,email,pass,picture,created) values(?,?,?,?,now());');
    $statement->bindParam(1, $name,PDO::PARAM_STR);
    $statement->bindParam(2, $email,PDO::PARAM_STR);
    $statement->bindParam(3, $pass,PDO::PARAM_STR);
    $statement->bindParam(4, $image,PDO::PARAM_STR);
    $statement->execute();
    echo "Insert 成功";
} catch(PDOException $e) {
    echo "Insert 失敗";
}
```

同様にユーザ2, 3も作りましょう。

メッセージを3つ、三人のユーザがそれぞれ書き込み、1つ目は人気ですが、3つ目は人気がないというデータを作ります

```

-- メッセージ
insert into messages (message,mid,mecre,deleted) values(' 1 回目のメッセージです',1,now(),false);
insert into messages (message,mid,mecre,deleted) values(' 2 回目のメッセージです',2,now(),false);
insert into messages (message,mid,mecre,deleted) values(' 3 回目のメッセージです',3,now(),false);

-- メッセージにいいね
insert into melike values (1,2);
insert into melike values (1,3);
insert into melike values (2,3);

-- メッセージにコメント
insert into comments (meid,mid,comment,cocre) values (1,2,' 1 回目に 1 回目のコメント',now());
insert into comments (meid,mid,comment,cocre) values (2,1,' 2 回目に 1 回目のコメント',now());
insert into comments (meid,mid,comment,cocre) values (1,1,' 1 回目に 2 回目のコメント',now());
insert into comments (meid,mid,comment,cocre) values (1,3,' 1 回目に 3 回目のコメント',now());

-- コメントにいいね
insert into colike values (1,1);
insert into colike values (1,3);
insert into colike values (2,3);

```

図 6-7 テストデータの SQL コマンド

では、このデータからデータ一覧を取得してみましょう。SQL 文を考えます。まずは、メッセージの一覧を取得しましょう。メッセージは誰が書いたか、ID は何番か、メッセージの内容、そしてコメントの件数といいねの件数を、削除していないもので、最新のものから逆順に取得します。

```

select s1.mid,s1.meid,s1.message,mb.picture,mb.mname,s1.mecre,
COALESCE((select count(*) from comments as s2 where s1.meid=s2.meid group by meid),0)
as comcnt,
COALESCE((select count(*) from melike as s3 where s1.meid=s3.meid group by meid),0) as
likecnt
from messages as s1
inner join members mb on s1.mid=mb.mid
where s1.deleted=false
order by s1.meid desc

```

図 6-8 メッセージ一覧の SQL

COALESCE は NULL が予想される項目で NULL 以外が欲しいときに使用する関数です。ここではコメントが無い場合やいいねが無い場合に NULL にならないように 0 に変換しています。また、サブクエリを使ってク

エリの中にクエリを記述し、内側のクエリと外側のクエリを関連付けることで、それぞれの条件での件数
を取得しています。

実行してみましょう。

mid	meid	message	picture	mname	mecre	comcnt	likecnt
3	3	3つ目のメッセージです	noimage.jpg	user3	2025-02-07 00:14:00	0	0
2	2	2つ目のメッセージです	noimage.jpg	user2	2025-02-07 00:14:00	1	1
1	1	1つ目のメッセージです	noimage.jpg	user1	2025-02-07 00:14:00	3	2

図 6-9 メッセージ一覧の結果

次に、コメントの一覧を取得しましょう。コメントは同じメッセージに対するコメントでメッセージが
削除されていないものの中で、メッセージとコメントの書き込みの逆順で取得します。コメントが無い場
合は出力しません。

```
select co.cid,co.comment,co.meid,co.mid,co.cocre,mb.mname from
(comments co inner join messages me on co.meid=me.meid )
inner join members mb on mb.mid=co.mid
where me.deleted=false
order by meid desc,cid desc;
```

図 6-10 コメント一覧の SQL

cid	comment	meid	mid	cocre	mname
2	2つ目に1つ目のコメント	2	1	2025-02-07 00:14:00	user1
4	1つ目に3つ目のコメント	1	3	2025-02-07 00:14:00	user3
3	1つ目に2つ目のコメント	1	1	2025-02-07 00:14:00	user1
1	1つ目に1つ目のコメント	1	2	2025-02-07 00:14:00	user2

図 6-11 コメント一覧の結果

Meid=3 のコメントは無いので出力されていません。

では、この SQL とひな型をつないで、現在のデータを index.php で実行してみましょう。sampleindex.html
を引用して作りましょう。この際、78 行から 117 行は繰り返し部分なので削除してください。

【index.php 冒頭部】

```

<?php
session_start();
require('dbconnect.php');

//login.php 経由でない場合は LOGINに戻る
if (!isset($_SESSION['id'])) {
    header('Location: login.php');
    exit();
}

try {
    //全体のメッセージの一覧
    $posts = $db->query('select
s1.mid,s1.meid,s1.message,mb.picture,mb.mname,s1.mecre, ' .
        'COALESCE((select count(*) from comments as s2 where s1.meid=s2.meid
group by meid),0) as comcnt, ' .
        'COALESCE((select count(*) from melike as s3 where s1.meid=s3.meid
group by meid),0) as likecnt ' .
        'from messages as s1 ' .
        'inner join members mb on s1.mid=mb.mid ' .
        'where s1.deleted=false ' .
        'order by s1.meid desc;');

    //全体のコメントの一覧
    $comments = $db->query('select
co.cid,co.comment,co.meid,co.mid,co.cocre,mb.mname from (comments co ' .
        'inner join messages me on co.meid=me.meid ) ' .
        'inner join members mb on mb.mid=co.mid ' .
        'where me.deleted=false ' .
        'order by meid desc,cid desc;');

    $comment = $comments->fetchAll(PDO::FETCH_ASSOC);
    //自分がメッセージに押したいいいねの一覧
    $mlikes = $db->query('select meid from melike ' .
        'where mid=' . $_SESSION['id']
    );

    $mlike = $mlikes->fetchAll(PDO::FETCH_ASSOC);
    //自分がコメント押したいいいねの一覧
    $clikes = $db->query('select cid from colike ' .
        'where mid=' . $_SESSION['id']
    );

    $clike = $clikes->fetchAll(PDO::FETCH_ASSOC);
} catch(PDOException $e) {
    $error['select'] = $e->getMessage();
}

$commentRow = 0; //コメントの行番号
?>

```

先に作った SQL とは別に、自分がメッセージに押したいいいねと、自分がコメントにいいねを押したコメ

ントの配列を取得する SQL も実行します。

【index.php head 内】

```
<?php
function searchMyLike(array $array, string $key, string $value): array {
    $result = array_filter($array, function ($item) use ($key, $value) {
        return is_array($item) && isset($item[$key]) && $item[$key] ===
$value;
    });

    return $result;
}
?>
</head>
```

ログインした人がいいねを入れたかどうか調べるための関数を作っておきます。各コメントのハートマークは自分がいいねを入れたものだけ赤く塗りつぶします。

【index.php メッセージ書き込み部】

```
<body>
    <div class="board-container">
        <div class="mess">
            <?php if (!empty($error['select'])) : ?>
            <p class="error">
            <?php echo $error['select'];?>
            </p>
            <?php endif; ?>
            <form action="" method="post">
                <textarea name="message" rows="5" placeholder="今の気分は？
"></textarea>
                <div class="mess-actions">
                    <button type="submit">投稿</button>
                </div>
            </form>
        </div>
```

【index.php post 繰り返し部】

```
<?php
foreach($posts as $post) :
?>
<div class="post">
    <div class="post-header">
        
        <span>投稿者: <?php echo h($post['mname']); ?></span>
        <span><time datetime="<?php echo h($post['mcre']); ?>"><?php
echo h($post['mcre']); ?></time></span>
        <div class="post-info">
            <span>
                <button class="pushDelete"><i class="fa-solid fa-trash
warning"></i></button>
            </span>
        </div>
    </div>
    <div class="post-content">
        <?php echo h($post['message']); ?>
    </div>
    .
<省略>
    .
        </div>
    </div>
    <?php
endforeach;
?>
</div>
```

\$posts で取得したメッセージの一覧から、メッセージの情報を出力します。投稿者の写真、名前、時間、メッセージを表示します。また、自分自身の書き込みの場合はメッセージを削除扱いにできるアイコンを出力します。

ここまで出来たら、メッセージが出力されるか実験してみてください。うまく行ったら、次のページのコメント等に進みましょう。

【index.php コメント書き込み部】

```
<div class="post-actions">
    <?php if (!empty(searchMyLike($mlike,"meid",$post['meid']))): ?>
        <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
            <i class="fa-solid fa-heart fa-red" id="likemark<?php echo
$post['meid']; ?>"></i></button>
        <?php else: ?>
            <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
            <i class="fa-regular fa-heart" id="likemark<?php echo
$post['meid']; ?>"></i></button>
        <?php endif; ?>
        <span class="like-count" id="likecount<?php echo
$post['meid']; ?>"><?php echo h($post['likecnt']); ?></span>
        <button class="pushComment" onclick="reply_open(<?php echo
h($post['meid']); ?>);">コメント</button>
        <div class="modal">
            <div class="replies" id="post<?php echo
h($post['meid']); ?>">
                <form action="" method="post">
                    <textarea name="reply" rows="10" placeholder="コメント
を入力してください"></textarea>
                    <button type="submit">書き込み</button>
                    <button onclick="reply_open(<?php echo
($post['meid']); ?>);return false;">キャンセル</button>
                    <input type="hidden" name="hid" value="<?php echo
h($post['meid']); ?>">
                </form>
            </div>
        </div>
    </div>
```

モーダルの実験で行ったように modal クラスの中に form を入れておきます。

【index.php コメント表示部分】

```

<div class="comments">
    <?php for($c = 0; $c < $post['comcnt']; $c++): ?>
        <div class="comment">
            <div class="comment-header">
                <span>コメント者: <?php echo
h($comment[$commentRow]['mname']); ?></span>
                <span><time datetime="<?php echo
h($comment[$commentRow]['cocre']); ?>"><?php echo
h($comment[$commentRow]['cocre']); ?></time></span>
            </div>
            <div class="comment-content">
                <?php echo h($comment[$commentRow]['comment']); ?>
            </div>
            <div class="comment-actions">
                <?php if
(!empty(searchMyLike($clike,"cid",$comment[$commentRow]['cid']))): ?>
                    <button class="pushComLike" data-cid="<?php echo
$comment[$commentRow]['cid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
                        <i class="fa-solid fa-heart fa-red"
id="clikemark<?php echo $comment[$commentRow]['cid']; ?>"></i>
                    <?php else: ?>
                        <button class="pushComLike" data-cid="<?php echo
$comment[$commentRow]['cid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
                            <i class="fa-regular fa-heart" id="clikemark<?php
echo $comment[$commentRow]['cid']; ?>"></i>
                        <?php endif; ?>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

comments クラスの中にコメントの数だけ comment クラスを作ります。表示したコメントに自分がいいねを入れていれば、ハートを赤く塗りつぶして表示します。

data-id や data-meid, data-cid は data 属性というもので data-xx の xx の部分は自由に設定できます。これは後で fetchAPI で使用します。

【index.php コメント、ポスト繰り返し部】

```

        <?php
            $commentRow++;
        endfor;
    ?>
</div>
</div>
<?php
endforeach;
?>
</div>

```

最初にコメントの一覧を取得しているので、該当のメッセージに該当の数だけコメントを出力します。

【index.php script 部】

```
<script src="js/fetch.js"></script>
<script>
  function reply_open(id) {
    var rep = document.getElementById("post"+id);
    rep.style.display = rep.style.display == "block" ? "none" : "flow";
    var parent = rep.parentNode;
    parent.style.display = parent.style.display == "block" ? "none" :
"flow";
  }

```

ログアウト

今の気分は？

投稿



投稿者: user1 2025-02-10 01:05:28 

プログラムからの入力

♡ 1

コメント

コメント者: user2

2025-02-12 10:41:32

USER2からのコメント

♡

コメント者: user1

2025-02-12 00:53:40

プログラムからのコメント

♡



投稿者: user3 2025-02-07 00:14:00

3つ目のメッセージです

♡ 0

コメント



投稿者: user2 2025-02-07 00:14:00

2つ目のメッセージです

♡ 1

コメント

コメント者: user1

2025-02-07 00:14:00

2つ目に1つ目のコメント

♡

図 6-12 データー一覧表示

メッセージ書き込み

一覧が見えるようになりましたので、流れに沿ってメッセージの書き込みを行いましょう。実験で出来ていますので、databasetest.php からコピーして取り込みましょう。

【index.php 追加部分】

```
header('Location: login.php');
exit();
}

//POST された場合
if (!empty($_POST)) {
    //メッセージの書き込み
    if (!empty($_POST['message'])) {
        $message = h($_POST['message']);
        $mid = $_SESSION['id'];
        if ($message!='') {
            try {
                $statement = $db->prepare('insert into messages
(message,mid,mecre,deleted) values(?,?,now(),false);');
                $statement->bindParam(1, $message,PDO::PARAM_STR);
                $statement->bindParam(2, $mid,PDO::PARAM_INT);
                $statement->execute();
            } catch(PDOException $e) {
                $error['insert'] = "書き込みに失敗しました。";
            }
        }
    }
}
```

コメント入力・保存

モーダルは実験で出来ているので、コメントの書き込みを追加しましょう。コメントの書き込みは formで行いますので、メッセージとほぼ同じ流れです。

【index.php メッセージ書き込みに続いて】

```

    } //<メッセージへの書き込みに続いて>
    //コメント書き込み
    } elseif (!empty($_POST['reply'])) {
        $reply = h($_POST['reply']);
        $meid = $_POST['hid'];
        $mid = $_SESSION['id'];
        if ($reply!='') {
            try {
                $statement = $db->prepare('insert into comments
(meid,mid,comment,cocre) values(?,?,?,now());');
                $statement->bindParam(1, $meid,PDO::PARAM_INT);
                $statement->bindParam(2, $mid,PDO::PARAM_INT);
                $statement->bindParam(3, $reply,PDO::PARAM_STR);
                $statement->execute();
            } catch(PDOException $e) {
                $error['insert'] = "書き込みに失敗しました。";
            }
        }
    }
}

```

メッセージ同様にコメントを書き込みます。場所は、メッセージの書き込みの if に elseif を追加するところです。Comments テーブルに、どのメッセージかの番号と誰が書いたかの ID とコメントを書き込みます。どのメッセージに対してかは hidden で hid で POST されています。

【index.php pushComment 部分(like-count の後)】

```

        <span class="like-count" id="likecount">?php echo
$post['meid']; ?><?php echo h($post['likecnt']); ?></span>
        <button class="pushComment" onclick="reply_open(?php echo
h($post['meid']); ?>);">コメント</button>
        <div class="modal">
            <div class="replies" id="post">?php echo
h($post['meid']); ?><div class="reply-form">
                <form action="" method="post">
                    <textarea name="reply" rows="10" placeholder="コメント
を入力してください"></textarea>
                    <button type="submit">書き込み</button>
                    <button onclick="reply_open(?php echo
($post['meid']); ?>);return false;">キャンセル</button>
                    <input type="hidden" name="hid" value="?php echo
h($post['meid']); ?>">
                </form>
            </div>
        </div>
    </div>

```

削除機能

削除に関しては、ゴミ箱アイコンのクリックで行います。ただしデータベースから削除するのではなく、deleted フラグを立てて、実体は消さずに消したことにする方法をやってみましょう。いままでのメッセー

ジの一覧のとり方もこれに対応した SQL になっています。メッセージの削除の場所はコメントの書き込みの後ろに elseif を追加するところです。

【index.php コメント書き込みに続いて】

```
    }
    //メッセージ削除
} elseif (!empty($_POST['delmess'])) {
    $meid = $_POST['delmess'];
    try {
        $statement = $db->prepare('update messages set deleted=true where
meid=?;');
        $statement->bindParam(1, $meid, PDO::PARAM_INT);
        $statement->execute();
    } catch(PDOException $e) {
        $error['insert'] = "削除に失敗しました。";
    }
}
```

【index.php post-info の内部】

```
<div class="post-info">
    <span>
        <?php if ($post['mid'] == $_SESSION['id']): ?>
        <form action="" method="post" onsubmit="return confirm('削除
してよろしいですか?');">
            <button class="pushDelete"><i class="fa-solid fa-trash
warning"></i></button>
            <input type="hidden" name="delmess" value="<?php echo
$post['meid'];?>">
        </form>
        <?php endif; ?>
    </span>
</div>
```

Form に onsubmit を付けているので「削除してもよろしいですか？」で OK した時だけ削除が実行されます。delmess に指定したメッセージ番号の deleted が true になります。

ところでこの form には submit がありません。Onsubmit はありますが、これでは form が POST されません。その理由は、削除なのでいきなり submit しなくなかったからです。そこで pushDelete クラスの button に対して、javascript で submit を実行するイベントを登録したいと思います。

【index.php script の中に追加】

```
<script>
    document.querySelectorAll(".pushDelete").forEach(button => {
        button.addEventListener("click", function() {
            this.submit();
        });
    });
});
```

また css も追加します。

【css/style.css 削除系】

```
.post-info {
    font-size: .75em;
    text-align: right;
    margin-left: 10px;
}

.post-info .pushDelete {
    border: none;
    background-color: white;
    cursor: pointer;
}

.post-info i {
    padding: 0 5px 0 10px;
}
```

■ メッセージへのいいね

メッセージへのいいねは、メッセージやコメントの書き込みと違い、書き換わる部分がハートの横の数と、ハートの色です。あらかじめ試しておきたいで実験した、いいねと同じ様に Fetch API を使用します。なので JavaScript と PHP を連動させて処理を行い、ページの全再表示は行わず、関連する場所だけ書き換えます。

メッセージのいいねはログインした本人がそのメッセージにいいねを入れたかどうかを表しています。なので最初で本人がいいねを入れたメッセージの配列を予め読んでおいて、メッセージを表示する時に、そのメッセージに本人がいいねしたかを関数で調べて、ハートの色を切り替えています。

【index.php メッセージの post-actions の中】

```

</div>
<div class="post-actions">
    <?php if (!empty(searchMyLike($mlike,"meid",$post['meid']))): ?>
        <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
            <i class="fa-solid fa-heart fa-red" id="likemark<?php echo
$post['meid']; ?>"></i></button>
        <?php else: ?>
            <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
                <i class="fa-regular fa-heart" id="likemark<?php echo
$post['meid']; ?>"></i></button>
            <?php endif; ?>
            <span class="like-count" id="likecount<?php echo
$post['meid']; ?>"><?php echo h($post['likecnt']); ?></span>

```

ハートのアイコンには"likemark"+メッセージ番号で id が振ってあります。Fetch API でそのアイコンのクラスの書き換えと"likecount"+メッセージ番号で振られた id の場所にいいねの個数が表示されます。その呼び出しの関数名は pushLikeButton ですが、ここでは onclick で宣言せずに、プログラム下部の script でアイコンの親タブの button に click イベントを追加しています。その button にはいいねのメッセージ番号や自分の ID を"data-*"のデータ属性で記入してあります。これを読みこんで pushLikeButton 関数を呼び出しています。削除の時は submit を出しましたが、今回は submit は行わず、こっそりと裏から javascript と php を使ってデータベースを書き換えと、ハートの色と数だけ書き換えます。

【index.php script の中に追加】

```

document.querySelectorAll(".pushLike").forEach(button => {
    button.addEventListener("click", function() {
        const meid = this.getAttribute("data-meid");
        const id = this.getAttribute("data-id");
        pushLikeButton(meid,id,'likecount'+meid,
            'likemark'+meid);
    });
});
// 更新関数呼び出し
async function pushLikeButton(mess,member,ltag,itag) {
    let outicon = document.getElementById(itag);
    let list = outicon.className;
    if (list.indexOf('fa-red')== -1) { //赤が無ければ現在 0 なので 1 に変えたい
        var onoff = 1;
    } else {
        var onoff = 0;
    }
    await getLike( mess,member,onoff,ltag, itag );
}

```


この JavaScript 関数を/body の上に追加記述します。関数は呼び出されたら、指定されたアイコンの id のクラスの中に fa-red つまり赤いハートで表示しているかを取得します。赤がなければ自分のいいねを追加、赤ならばいいねを削除する関数 getLike() を呼びます。

getLike を呼び出すのに await を付けています。これは async が付いた関数を呼び出すときの作法で、非同期処理、つまり少し時間が掛かっても、結果が返ってきたら動き始める処理の時に使います。JavaScript から PHP を呼び出すには間にネットワークやデータベースのやり取りなどがあり、人間の感覚ではすぐですか、コンピュータにとっては膨大な時間が過ぎていきます。その時間が過ぎてもしっかりと結果を受け取るようにするにはこのような await と async を使った処理を行います。この pushLikeButton 関数で書き換え処理をしてしまうと、実はページの始めに読み取ったデータベースのいいねの配列の情報と実体がずれてしまいます。なので色の方を信用するとしていいねの色を処理します。

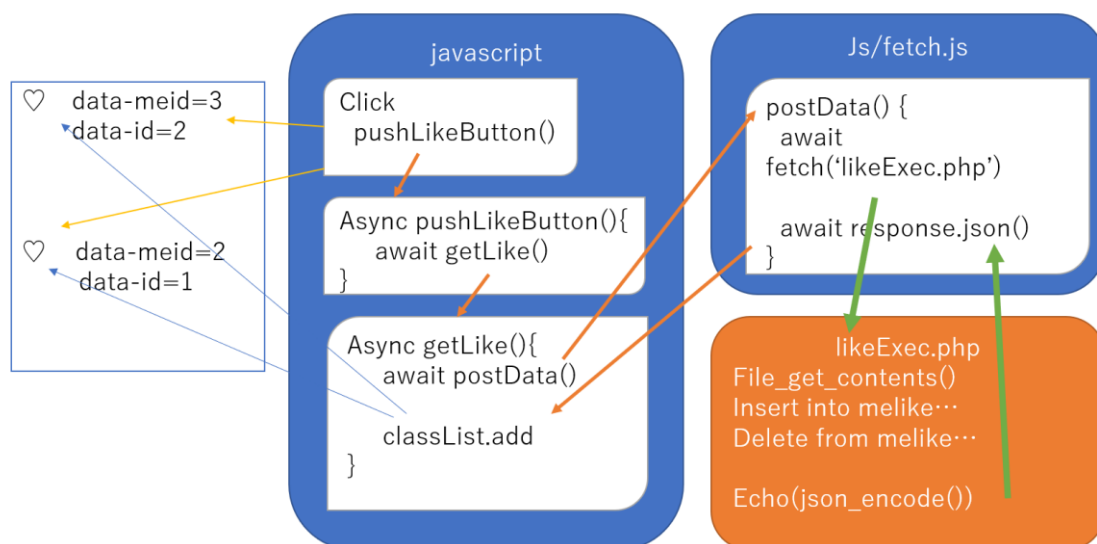


図 6-13 fetchAPI の連動

【index.php script の中に追加】

```
// いいね更新関数
async function getLike( mess, member, onoff, outTag, markTag ) {
  const data = {
    meid : mess,
    mid  : member,
    on   : onoff
  };
  try {
    const res = await postData( 'likeExec.php' , data );
    if ( !res.status ) {
      // データの取得状態のチェック
      throw res.retData ;
    }
    // データが正常に取得できた処理
    const retData = res.retData.data ;
    if ( res.retData.status ) { // リク
      エストのエラーチェック
      const lcnt = `${retData.likecount}` ;
      var outdiv = document.getElementById(outTag);
      outdiv.innerHTML = lcnt;
      var outicon = document.getElementById(markTag);
      if (onoff==1) { //on にする
        outicon.classList.replace('fa-regular','fa-solid');
        outicon.classList.add('fa-red');
      } else {
        outicon.classList.replace('fa-solid','fa-regular');
        outicon.classList.remove('fa-red');
      }
    } else {
      console.error( retData.reason ) ;
    }
  } catch( e ) { // エラーの処理
    // console.error(e) ;
  }
}
```

getLike()はデータベースの更新処理をした後に、その後のいいねの数を再取得して返しています。ところがこのようなデータベースを扱う処理はJavaScriptではできません。そのため、postData という汎用の PHP 呼び出しの関数を使って PHP を呼び出しています。

【js/fetch.js】

```
// 汎用 JSON 取得関数
async function postData( argURL , argData ) {
  try {
    const response = await fetch(
      argURL , {
        method: 'POST' , // メソッド POST
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify( argData ), // JSON でパラメータ
      }
    );
    if ( !response.ok ) { // fetch のエラーチェック
      throw 'FETCH ERROR! HTTP Status:' + response.status ; // fetch でエラーがあればエラーを投げる
    }
    const resData = await response.json() ; // レスポンスデータの取得
    return { // データの取得状態(true)と取得した JSON データを返します。
      status: true ,
      retData: resData ,
    } ;
  } catch( e ) { // エラー処理
    return { // データの取得状態(false)とエラー内容を返します。
      status: false ,
      retData: e ,
    } ;
  }
}
```

また、この処理を取り込んでおくために、/body の前に js フォルダの fetch.js を読み込んでおきます。

```
</div>
<script src="js/fetch.js"></script>
<script>
```

postData でやっているのが、Fetch API の呼び出しです。ここで likeExec.php を呼び出して、その結果を json で受け取り、JavaScript に渡します。

【likeExec.php】

```
<?php
require('dbconnect.php');

//like の数を取得する
$retAry = array() ; // 返信用配列データ
$req = json_decode( file_get_contents( 'php://input' ), true ) ; // リクエスト
データを取得 *注 1
$meid = $req[ 'meid' ] ; // リクエストされた ID
$mid = $req[ 'mid' ] ; // リクエストされた ID
$flag = $req[ 'on' ] ; // リクエストされた ID

// //Like を追加
if ($flag==true) {
    $istatement = $db->prepare('insert into melike values (?,?);');
    $istatement->bindParam(1, $meid, PDO::PARAM_INT);
    $istatement->bindParam(2, $mid, PDO::PARAM_INT);
    $istatement->execute();
} else {
    $istatement = $db->prepare('delete from melike where meid=? and mid=?;');
    $istatement->bindParam(1, $meid, PDO::PARAM_INT);
    $istatement->bindParam(2, $mid, PDO::PARAM_INT);
    $istatement->execute();
}

//Like の個数取得
$gstatement = $db->prepare('select count(*) as lcnt from melike where
meid=?;');
$gstatement->bindParam(1,$meid,PDO::PARAM_INT);
$gstatement->execute();
$cnt = $gstatement->fetch();
$lcnt = $cnt['lcnt'];

// プロフィールデータ
$likeAry = array(
    'likecount' => $lcnt ) ;
$retAry = array() ; // 返信用配列データ
$req = json_decode( file_get_contents( 'php://input' ), true ) ; // リクエスト
データを取得 *注 1
$retAry[ 'status' ] = true ; // プロフィール取得成功
$retAry[ 'data' ] = $likeAry ; // プロフィールの設定

echo( json_encode( $retAry ) ) ; // JSON 形式でデータを返します。
?>
```

呼び出された likeExec.php は JavaScript からの POST データを受けて、引数を分解して、それぞれの処理を行います。

メッセージへのいいねを追加する場合は、melike へ insert、いいねを解除する場合は melike の delete を行います。そのあといいねの件数を select で取得してそのデータを json にして echo します。PHP の

echo を今度は JavaScript がレスポンスとして受け取って、getLike()の retdata に渡ります。このようにして JavaScript と PHP がデータや処理のやり取りをするとクライアント側からサーバー側の操作を行うことができます。

コメントへのいいね

今度はコメントへのいいねです。メッセージへのいいねとほぼ同じことをしますが、コメントの時はいいねの件数を返さないところが違います。

【index.php post】

```
</div>
<div class="post-actions">
    <?php if (!empty(searchMyLike($mlike,"meid",$post['meid']))): ?>
        <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
            <i class="fa-solid fa-heart fa-red" id="likemark<?php echo
$post['meid']; ?>"></i></button>
        <?php else: ?>
            <button class="pushLike" data-meid="<?php echo
$post['meid'];?>" data-id="<?php echo $_SESSION['id']; ?>">
                <i class="fa-regular fa-heart" id="likemark<?php echo
$post['meid']; ?>"></i></button>
            <?php endif; ?>
            <span class="like-count" id="likecount<?php echo
$post['meid']; ?>"><?php echo h($post['likecnt']); ?></span>
```

【index.php script へ追加】

```
document.querySelectorAll(".pushComLike").forEach(button =>
{
    button.addEventListener("click", function() {
        const cid = this.getAttribute("data-cid");
        const id = this.getAttribute("data-id");
        pushComLikeButton(cid,id,'clikemark'+cid);
    });
});
// コメントいいね更新関数呼び出し
async function pushComLikeButton(cid,member,itag) {
    let outicon = document.getElementById(itag);
    let list = outicon.className;
    if (list.indexOf('fa-red')== -1) { //赤が無ければ現在 0 なので 1 に変えたい
        var onoff = 1;
    } else {
        var onoff = 0;
    }
    await getComLike( cid,member,onoff, itag );
}
```

コメントの ID とメンバーID とアイコンの ID 名を渡します。

【index.php script に追加】

```
// コメントいいね更新関数
async function getComLike( cid, member, onoff, markTag ) {
  const data = {
    cid : cid,
    mid : member,
    on   : onoff
  } ;
  try {
    const res = await postData( 'clikeExec.php' , data) ;
    if ( !res.status ) {                                     // デ
一タの取得状態のチェック
      throw res.retData ;                                   // 正
常に取得できなかったらエラーを投げて catch()で処理する。
    }
    // データが正常に取得できた処理
    const retData = res.retData.data ;
    if ( res.retData.status ) {
      var outicon =
document.getElementById(markTag);                          // リクエストのエラーチ
エック
      if (onoff==1) { //onにする
        outicon.classList.replace('fa-regular','fa-solid');
        outicon.classList.add('fa-red');
      } else {
        outicon.classList.replace('fa-solid','fa-regular');
        outicon.classList.remove('fa-red');
      }
      // プロフィールの表示
    } else {
      console.error( retData.reason ) ;                     // リ
クエストエラーの表示
    }
  } catch( e ) {                                           // エラ
一の処理
    // console.error(e) ;
  }
}
```

メッセージの時と同様に php を呼び出します。clikeExec.php という php で処理します。

【clikeExec.php】

```
<?php
require('dbconnect.php');

//like の数を取得する
$retAry = array() ; // 返信用配列データ
$req = json_decode( file_get_contents( 'php://input' ), true ) ; // リクエスト
データを取得 *注 1
$cid = $req[ 'cid' ] ; // リクエストされた ID
$mid = $req[ 'mid' ] ; // リクエストされた ID
$flag = $req[ 'on' ] ; // リクエストされた ID

//Like を追加
if ($flag==true) {
    $istatement = $db->prepare('insert into colike values (?,?)');
    $istatement->bindParam(1, $cid, PDO::PARAM_INT);
    $istatement->bindParam(2, $mid, PDO::PARAM_INT);
    $istatement->execute();
} else {
    $istatement = $db->prepare('delete from colike where cid=? and mid=?');
    $istatement->bindParam(1, $cid, PDO::PARAM_INT);
    $istatement->bindParam(2, $mid, PDO::PARAM_INT);
    $istatement->execute();
}

// プロフィールデータ
$likeAry = array();
$retAry = array() ; // 返信用配列データ
$req = json_decode( file_get_contents( 'php://input' ), true ) ; // リクエスト
データを取得 *注 1
$retAry[ 'status' ] = true ; // プロフィール取得成功
$retAry[ 'data' ] = $likeAry ; // プロフィールの設定

echo( json_encode( $retAry ) ) ; // JSON 形式でデータを返しま
す。
?>
```

コメントへのいいねは colike テーブルに行います。いいねを追加するときは colike に insert、いいねを解除するときは colike を delete します。件数は返さないでステータスだけ返し、中身は今のところ空を返しています。

ログアウト

ログインして投稿したらログアウトの部分を作しましょう
ここまでできたらようやく一通り処理がそろったことになります

7. ログアウト

セッション削除

最後にログアウト時にセッションを削除します。

【index.php board-container 直下】

```
<div class="board-container">

    <div class="exit"><a href="logout.php" onclick="return confirm('logout
    します');">ログアウト</a></div>

    <div class="mess">
```

【css/style.css 投稿の前に追加】

```
.exit {
    display: flex;
    justify-content: flex-end;
    width: 100%;
}
.exit a {
    text-decoration: none;
    font-size: xx-small;
    color: red;
}
/* 投稿 */
```

【logout.php】

```
<?php
session_start();

$_SESSION = array();

session_destroy();

header('Location: login.php');
exit();
?>
```

セッションを削除して、loginに戻って終了します。

さて、ここまでパソコン画面の作成をしてきましたが、最初に言ったようにスマホ画面の対応も必要です。

そこで、@media を使った、スマホ画面の css をまとめておきましょう。

```
/* スマホ対応 */
@media (max-width: 600px) {
  .board-container,
  .check-container,
  .thanks-container {
    width: 98%;
    box-sizing: border-box;
    margin: 10px;
  }
  .mess {
    width: 100%;
  }
  .mess textarea {
    width: 100%;
  }
  .login-container {
    width: 100%; /* スマホのときの幅 */
    box-sizing: border-box; /* パディングとボーダーを含める */
    margin: 10px; /* 画面端から少し余裕を持たせる */
  }
  .login-container button {
    width: 98%;
  }
  .replies {
    display: flex;
    justify-content: flex-start;
  }
  .replies textarea {
    width: 100%;
  }
}
```

これで簡易掲示板の作成を終わります。掲示板はデータベースを利用する基礎ですので、これを元に色々改造してみるのがいいでしょう。

8. Index

あ

await ・105
async ・105
AMPPS ・6

く

クライアントサイド ・6

こ

compose.yaml ・8

さ

サーバーサイド言語 ・6

た

第1正規形 ・15
第2正規形 ・17
第3正規形 ・17

て

データベース正規化 ・14

と

Docker ・7
Dockerfile ・8

は

HASH 化 ・40

ひ

PhpMyAdmin ・30
非正規形 ・14

り

Linux ・6

れ

レスポンシブデザイン ・44